ICPC Notebook

template
gen.cpp 2
hash.sh 2
hash_values.txt 2
settings.sh 2
stress_test.sh 2
template.hpp 2
template_speedstar.hpp 2
vim_setteing.sh 2
data-structure
2DSegtree.hpp 2
ConvexHullTrick.hpp 3
ConvexHullTrick2.hpp 4
FastSet.hpp 4
LiChao.hpp 4
Mosalgo.hpp 5
RollbackUnionFind.hpp 5
UnionFind.hpp 5
fenwick_tree.hpp 5
lazy_segtree.hpp 5
segtree.hpp 6
math
BinaryGCD.hpp 7
EulerPhi.hpp 7
ExtGCD.hpp 7
Matrix.hpp 7
floor_sum.hpp 7
gensikon.hpp 8
lagrange.hpp 8
linear_congruence.hpp 8
mathtemplate.hpp 8
xorConvolution.hpp 8
modint
BarrettReduction.hpp 9
combination.hpp 9
convolution.hpp 9
modint.hpp 9
extra
modint_fast.hpp 9
FPS
FFT.hpp 10
FFT_fast.hpp 10
graph
CentroidDecomposition.cpp 10
TreeHash.hpp 11
Trie.hpp 11
Trie_yokohama.hpp 11
dijkstra.hpp 11
edge.hpp 11
eulertrail.hpp 12
gyouretsukiteiri.hpp 12
hld.hpp 12
lca.hpp 13
lca_me.hpp 13
lowlink.hpp 14
scc.hpp 14
scc_yokohama.hpp 14
flow
bipartite-matching.hpp 15
maxflow.hpp 15
mincostflow.hpp 15
燃やす埋める.md 16
string
KMP.hpp 16
Manacher.hpp 16
RollingHash.hpp 16
SuffixArray_spped_star.hpp 16
Zalgorithm.hpp 16
kaibun.hpp 17
lcp_array.hpp 17
rolling_hash.hpp 17
suffix_array.hpp 17
z_algorithm.hpp 17
geometry
memo
Bi-Connected-Components.hpp 18
Dynamic-Li-Chao-Tree.hpp 19
Primes.md 19
bostanmori.hpp 20
dynamic_convex_hull_trick.hpp 23
dynamic_segtree.hpp 24
fps.hpp 24
geometry.hpp 27
int128.hpp 30
random.hpp 30
range_edge_graph.hpp 30

template

gen.cpp

md5: 28f8b5

```cpp
int main(){cin.tie(0);ios::sync_with_stdio(false);
    // 実行ごとに異なる強力なシードを生成
    random_device rd;
    mt19937 mt(rd());

    // 例: 1から100までのランダムな整数
    int T=1;
    uniform_int_distribution<int> randN(1, 5);
    uniform_int_distribution<int> randV(1, 7);
    int K=randV(mt);

    cout<< T << endl;
    cout<< N <<" " << K <<  endl;
    while(1){
        vi a;
        rep(i,N)a.push_back(randV(mt));
        sort(all(a));
        a.erase(unique(all(a)),a.end());
        if(a.size()==N){
            for(auto u:a)cout << u << " ";
            cout << endl;
            break;
        }
    }
}
```

hash.sh

```sh
# 使い方: sh hash.sh -> コピペ -> Ctrl + D
# コメント・空白・改行を削除して md5 でハッシュする。コンパイラに依存しない
perl -0777 -pe 's{//[^\n]*|/\*.*?\*/}{}gs' \
  | tr -d '[:space:]' \
  | { command -v md5sum > /dev/null 2>&1 && md5sum || md5; } \
  | cut -c-6
```

hash_values.txt

```
コマンド
tr -d ' \t\r\n' < ~/.vimrc | md5sum
```

settings.sh

```sh
# CLion の設定
Settings → Build → CMake → Reload CMake Project
add_compile_options(-D_GLIBCXX_DEBUG)
# Caps Lock を Ctrl に変更
setxkbmap -option ctrl:nocaps
```

stress_test.sh

```sh
for f in cf naive gen;do g++ $f.cpp -O2 -o $f||exit;done
for((i=1;;i++));do
./gen>in;./cf<in>o1||break;./naive<in>o2||break;cmp -s o1
o2||break;done
echo case $i;cat in o1 o2
```

template.hpp

md5: c48eb9

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define rep(i, n) for(ll i = 0; i <(n); i++)
#define repi(i, a, b) for(ll i = (a); i < (b); i++)
#define all(x) (x).begin(), (x).end()
#define arr(x) (x).rbegin(), (x).rend()
template<typename T, typename S> bool chmin(T& a, const S& b) {
return a > b ? a = b, 1 : 0; }
template<typename T, typename S> bool chmax(T& a, const S& b) {
return a < b ? a = b, 1 : 0; }
using vi = vector<int>;
using P = pair<ll, ll>;
using vvi = vector<vi>;
using vll = vector<ll>;
using vvll = vector<vll>;
using vp = vector<P>;
using i128 = __int128_t;
#define sz(a) (ll)(size(a))
const ll inf = 2e18;

void solve(int test) {}
int main() {
   cin.tie(0);
   ios::sync_with_stdio(false);
   int t = 1;  // cin >>t;
   rep(test, t) solve(test);
}
```

template_speedstar.hpp

md5: f368a0

```cpp
#include <bits/stdc++.h>
using namespace std;
#define ll long long
#define pii pair<int, int>
#define pll pair<ll, ll>
#define vi vector<int>
#define vl vector<ll>
#define ov4(a, b, c, d, name, ...) name
#define rep3(i, a, b, c) for(ll i = (a); i < (b); i += (c))
#define rep2(i, a, b) rep3(i, a, b, 1)
#define rep1(i, n) rep2(i, 0, n)
#define rep0(n) rep1(aaaaa, n)
#define rep(...) ov4(__VA_ARGS__, rep3, rep2, rep1, rep0)
(__VA_ARGS__)
#define per(i, a, b) for(ll i = (a) - 1; i >= (b); i--)
#define fore(e, v) for(auto&& e : v)
#define all(a) begin(a), end(a)
#define si(a) (int)(size(a))
#define lb(v, x) (lower_bound(all(v), x) - begin(v))
#define eb emplace_back

template<typename T, typename S> bool chmin(T& a, const S& b) {
return a > b ? a = b, 1 : 0; }
template<typename T, typename S> bool chmax(T& a, const S& b) {
return a < b ? a = b, 1 : 0; }

const int INF = 1e9 + 100;
const ll INFL = 3e18 + 100;

#define i128 __int128_t

struct _ {
   _() { cin.tie(0)->sync_with_stdio(0), cout.tie(0); }
} __;
```

vim_setteing.sh

```sh
sy on
se nu bs=2 cin sta ts=4 sw=4 et ic scs hls is cb=unnamed
let g:loaded_matchparen=1
ino ( ()<Left>
ino { {}<Left>
ino [ []<Left>
ino <expr> ) getline('.')[col('.')-1]==')'?'<Right>':')'
ino <expr> } getline('.')[col('.')-1]=='}'?'<Right>':'}'
ino <expr> ] getline('.')[col('.')-1]==']'?'<Right>':']'
nn <Space> :noh<CR>
```

data-structure

2DSegtree.hpp

md5: 6ce929

```cpp
#pragma once

template<typename T, typename F> struct SegmentTree2D {
   private:
   int id(int h, int w) const { return h * 2 * W + w; }

   public:
   int H, W;
   vector<T> seg;
   const F f;
   const T I;

   SegmentTree2D(int h, int w, F _f, const T& i) : f(_f), I(i)
{ init(h, w); }

   void init(int h, int w) {
      H = W = 1;
      while(H < h) H <<= 1;
      while(W < w) W <<= 1;
```

```
      seg.assign(4 * H * W, I);
   }

   // build にのみ呼ぶ
   void set(int h, int w, const T& x) { seg[id(h + H, w + W)] =
x; }

   void build() {
      // w in [W, 2W)
      for(int w = W; w < 2 * W; w++) {
         for(int h = H - 1; h; h--) { seg[id(h, w)] =
f(seg[id(2 * h + 0, w)], seg[id(2 * h + 1, w)]); }
      }
      // h in [0, 2H)
      for(int h = 0; h < 2 * H; h++) {
         for(int w = W - 1; w; w--) { seg[id(h, w)] =
f(seg[id(h, 2 * w + 0)], seg[id(h, 2 * w + 1)]); }
      }
   }

   T get(int h, int w) const { return seg[id(h + H, w + W)]; }
   T operator()(int h, int w) const { return seg[id(h + H, w +
W)]; }

   void update(int h, int w, const T& x) {
      h += H, w += W;
      seg[id(h, w)] = x;
      for(int i = h >> 1; i; i >>= 1) { seg[id(i, w)] =
f(seg[id(2 * i + 0, w)], seg[id(2 * i + 1, w)]); }
      for(; h; h >>= 1) {
         for(int j = w >> 1; j; j >>= 1) { seg[id(h, j)] =
f(seg[id(h, 2 * j + 0)], seg[id(h, 2 * j + 1)]); }
      }
   }

   T _inner_query(int h, int w1, int w2) {
      T res = I;
      for(; w1 < w2; w1 >>= 1, w2 >>= 1) {
         if(w1 & 1) res = f(res, seg[id(h, w1)]), w1++;
         if(w2 & 1) --w2, res = f(res, seg[id(h, w2)]);
      }
      return res;
   }

   // [ (h1,w1), (h2,w2) ) 半開
   T query(int h1, int w1, int h2, int w2) {
      if(h1 >= h2 || w1 >= w2) return I;
      T res = I;
      h1 += H, h2 += H, w1 += W, w2 += W;
      for(; h1 < h2; h1 >>= 1, h2 >>= 1) {
         if(h1 & 1) res = f(res, _inner_query(h1, w1, w2)),
h1++;
         if(h2 & 1) --h2, res = f(res, _inner_query(h2, w1,
w2));
      }
      return res;
   }
};

/**
 * @brief 二次元セグメント木
 * @docs docs/data-structure-2d/2d-segment-tree.md
 */

// 使い方
int main() {
   cin.tie(0);
   ios::sync_with_stdio(false);
   int h, w, h1, w1, h2, w2;
   cin >> h >> w >> h1 >> w1 >> h2 >> w2;
   chmin(h2, h1);
   chmin(w2, w1);
   SegmentTree2D seg(h + 1, w + 1, [](ll a, ll b) { return
max(a, b); }, 0ll);
   vvll a(h + 1, vll(w + 1));
   repi(i, 1, h + 1) repi(j, 1, w + 1) cin >> a[i][j];
   vvll sum(h + 1, vll(w + 1)), sumt(h + 1, vll(w + 1)), suma(h
+ 1, vll(w + 1));
   repi(i, 1, h + 1) repi(j, 1, w + 1) sum[i][j] = sum[i][j -
1] + a[i][j];
   repi(j, 1, w + 1) repi(i, 1, h + 1) sum[i][j] += sum[i - 1]
[j];
   repi(i, h1, h + 1) repi(j, w1, w + 1) sumt[i][j] = sum[i][j]
- sum[i - h1][j] - sum[i][j - w1] + sum[i - h1][j - w1];
   repi(i, h2, h + 1) repi(j, w2, w + 1) suma[i][j] = sum[i][j]
- sum[i - h2][j] - sum[i][j - w2] + sum[i - h2][j - w2];
   repi(i, 1, h + 1) repi(j, 1, w + 1) seg.set(i, j, suma[i]
[j]);
   seg.build();
   ll ans = -inf;
   repi(i, h1, h + 1) repi(j, w1, w + 1) {
      ll now = sumt[i][j];
      now -= seg.query(i - (h1 - h2), j - (w1 - w2), i + 1, j +
1);
      chmax(ans, now);
   }
   cout << ans << endl;
}
```

ConvexHullTrick.hpp

md5: 84c189

```
// https://ei1333.github.io/library/structure/convex-hull-
trick/convex-hull-trick-add-monotone.hpp.html
template<typename T, bool isMin> struct
ConvexHullTrickAddMonotone {
#define F first
#define S second
   using P = pair<T, T>;
   deque<P> H;

   ConvexHullTrickAddMonotone() = default;

   bool empty() const { return H.empty(); }

   void clear() { H.clear(); }

   static constexpr int sgn(T x) { return x == 0 ? 0 : (x < 0 ?
-1 : 1); }

   static constexpr T floor_div(T n, T d) { return n / d - ((n
^ d) < 0 and n % d != 0); }

   static constexpr bool check(const P& a, const P& b, const P&
c) {
      if(b.S == a.S || c.S == b.S) return sgn(b.F - a.F) *
sgn(c.S - b.S) >= sgn(c.F - b.F) * sgn(b.S - a.S);
      // return (b.F-a.F)*(c.S-b.S) >= (b.S-a.S)*(c.F-b.F);
      if constexpr(is_integral<T>::value) {
         return floor_div(b.S - a.S, a.F - b.F) >=
floor_div(c.S - b.S, b.F - c.F);
      } else {
         return (b.F - a.F) * sgn(c.S - b.S) / abs(b.S - a.S)
>= (c.F - b.F) * sgn(b.S - a.S) / abs(c.S - b.S);
      }
   }

   void add(T a, T b) {
      if(!isMin) a *= -1, b *= -1;
      P line(a, b);
      if(empty()) {
         H.emplace_front(line);
         return;
      }
      if(H.front().F <= a) {
         if(H.front().F == a) {
            if(H.front().S <= b) return;
            H.pop_front();
         }
         while(H.size() >= 2 && check(line, H.front(), H[1]))
H.pop_front();
         H.emplace_front(line);
      } else {
         assert(a <= H.back().F);
         if(H.back().F == a) {
            if(H.back().S <= b) return;
            H.pop_back();
         }
         while(H.size() >= 2 && check(H[H.size() - 2],
H.back(), line)) H.pop_back();
         H.emplace_back(line);
      }
   }

   static constexpr T get_y(const P& a, const T& x) { return
a.F * x + a.S; }

   T query(T x) const {
      assert(!empty());
      int l = -1, r = H.size() - 1;
      while(l + 1 < r) {
         int m = (l + r) >> 1;
```

```
      if(get_y(H[m], x) >= get_y(H[m + 1], x)) l = m;
      else r = m;
    }
    if(isMin) return get_y(H[r], x);
    return -get_y(H[r], x);
  }

  T query_monotone_inc(T x) {
    assert(!empty());
    while(H.size() >= 2 && get_y(H.front(), x) >= get_y(H[1],
x)) H.pop_front();
    if(isMin) return get_y(H.front(), x);
    return -get_y(H.front(), x);
  }

  T query_monotone_dec(T x) {
    assert(!empty());
    while(H.size() >= 2 && get_y(H.back(), x) >=
get_y(H[H.size() - 2], x)) H.pop_back();
    if(isMin) return get_y(H.back(), x);
    return -get_y(H.back(), x);
  }

#undef F
#undef S
};
```

ConvexHullTrick2.hpp

md5: f180da

```
#define si(a) (int)(size(a))
#define pll pair<ll, ll>

//
https://github.com/KentaroMatsushita/icpc_library/blob/main/src/data-
structure/cht.hpp
template<bool isMin = true> struct CHT {
#define x first
#define y second
  CHT() = default;
  deque<pll> v;
  bool empty() { return v.empty(); }
  void clear() { return v.clear(); }
  inline int sgn(ll x) { return !x ? 0 : (x < 0 ? -1 : 1); }
  using D = long double;
  inline bool check(const pll& a, const pll& b, const pll& c)
{
    if(b.y == a.y or c.y == b.y) return sgn(b.x - a.x) *
sgn(c.y - b.y) >= sgn(c.x - b.x) * sgn(b.y - a.y);
    return D(b.x - a.x) * sgn(c.y - b.y) / D(abs(b.y - a.y))
>= D(c.x - b.x) * sgn(b.y - a.y) / D(abs(c.y - b.y));
  }
  void add(ll a, ll b) {
    if(!isMin) a *= -1, b *= -1;
    pll line(a, b);
    if(empty()) v.emplace_front(line);
    else {
      if(ll c = v[0].x; c <= a) {
        if(c == a) {
          if(v[0].y <= b) return;
          v.pop_front();
        }
        while(si(v) >= 2 and check(line, v[0], v[1]))
v.pop_front();
        v.emplace_front(line);
      } else {
        assert(a <= v.back().x);
        if(v.back().x == a) {
          if(v.back().y <= b) return;
          v.pop_back();
        }
        while(si(v) >= 2 and check(v[si(v) - 2], v.back(),
line)) v.pop_back();
        v.emplace_back(line);
      }
    }
  }
  ll get_y(const pll& a, const ll& x) { return a.x * x + a.y;
}
  ll query(ll x) {
    assert(!empty());
    int l = -1, r = si(v) - 1;
    while(l + 1 < r) {
      int m = (l + r) >> 1;
      if(get_y(v[m], x) >= get_y(v[m + 1], x)) l = m;
      else r = m;
    }
    return get_y(v[r], x) * (isMin ? 1 : -1);
  }
  ll query_monotone_inc(ll x) {
    assert(!empty());
    while(si(v) >= 2 and get_y(v[0], x) >= get_y(v[1], x))
v.pop_front();
    return get_y(v[0], x) * (isMin ? 1 : -1);
  }
  ll query_monotone_dec(ll x) {
    assert(!empty());
    while(si(v) >= 2 and get_y(v.back(), x) >= get_y(v.end()
[-2], x)) v.pop_back();
    return get_y(v.back(), x) * (isMin ? 1 : -1);
  }
#undef x
#undef y
};
```

FastSet.hpp

md5: 2cb8c9

```
// using u64 = uint64_t;
const u64 B = 64;
struct FastSet {
  u64 n;
  vector<vector<u64>> a;
  FastSet(u64 n_) : n(n_) {
    do a.emplace_back(n_ = (n_ + B - 1) / B);
    while(n_ > 1);
  }
  // bool operator[](ll i) const { return a[0][i / B] >> (i %
B) & 1; }
  void set(ll i) {
    for(auto& v : a) {
      v[i / B] |= 1ULL << (i % B);
      i /= B;
    }
  }
  void reset(ll i) {
    for(auto& v : a) {
      v[i / B] &= ~(1ULL << (i % B));
      if(v[i / B]) break;
      i /= B;
    }
  }
  ll next(ll i) {  // i を超える最小の要素
    rep(h, 0, sz(a)) {
      i++;
      if(i / B >= sz(a[h])) break;
      u64 d = a[h][i / B] >> (i % B);
      if(d) {
        i += countr_zero(d);
        while(h--) i = i * B + countr_zero(a[h][i]);
        return i;
      }
      i /= B;
    }
    return n;
  }
  ll prev(ll i) {  // i より小さい最大の要素
    rep(h, 0, sz(a)) {
      i--;
      if(i < 0) break;
      u64 d = a[h][i / B] << (~i % B);
      if(d) {
        i -= countl_zero(d);
        while(h--) i = i * B + __lg(a[h][i]);
        return i;
      }
      i /= B;
    }
    return -1;
  }
};
```

LiChao.hpp

md5: 92e522

```
struct LiChao {
  vll xs;
  vp t;
  LiChao(vll xs) : xs(xs), t(4 * sz(xs), P(0, inf)) {}
  ll f(P l, ll x) { return l.first * x + l.second; }
  void add(ll a, ll b) {
    P l(a, b);
    int k = 1, lo = 0, hi = sz(xs) - 1;
```

```
      while(1) {
         int m = (lo + hi) / 2;
         bool bl = f(l, xs[lo]) < f(t[k], xs[lo]), bm = f(l,
xs[m]) < f(t[k], xs[m]);
         if(bm) swap(t[k], l);
         if(lo == hi) return;
         if(bl != bm) hi = m, k = 2 * k;
         else lo = m + 1, k = 2 * k + 1;
      }
   }
   ll query(ll x) {
      int k = 1, lo = 0, hi = sz(xs) - 1;
      ll res = inf;
      while(1) {
         chmin(res, f(t[k], x));
         if(lo == hi) return res;
         int m = (lo + hi) / 2;
         if(x <= xs[m]) hi = m, k = 2 * k;
         else lo = m + 1, k = 2 * k + 1;
      }
   }
};
```

Mosalgo.hpp

md5: 6bff59

```
int n;                    // 数列の長さ
int q;                    // クエリ数
vector<int> l(q), r(q);   // クエリの[l,r)
/* 入力 */

// ソート
int sq = max(1, int(n / sqrt(q)));
vector<int> qi(q);
for(int i = 0; i < q; ++i) qi[i] = i;
sort(qi.begin(), qi.end(), [&](int i, int j) {  // クエリの順番を
pair(l/sq,r)でソート
   if(l[i] / sq != l[j] / sq) return l[i] < l[j];
   return r[i] < r[j];
});

// クエリの処理
// add(i)は追加操作、del(i)は削除操作
int nl = 0, nr = 0;  // [nl,nr)
for(int i : qi) {
   while(nl > l[i]) --nl, add(nl);
   while(nr < r[i]) add(nr), ++nr;
   while(nl < l[i]) del(nl), ++nl;
   while(nr > r[i]) --nr, del(nr);
   /* クエリの結果計算 */
}
```

RollbackUnionFind.hpp

md5: ef8861

```
#pragma once
#include <bits/stdc++.h>
using namespace std;

struct RollbackUnionFind {
   vector<int> data;
   stack<pair<int, int>> history;
   int inner_snap = 0;
   RollbackUnionFind(int n) { data.resize(n, -1); }
   int find(int x) { return data[x] < 0 ? x : find(data[x]); }
   bool unite(int x, int y) {
      x = find(x), y = find(y);
      history.push({x, data[x]});
      history.push({y, data[y]});
      if(x == y) return false;
      if(-data[x] < -data[y]) swap(x, y);
      data[x] += data[y];
      data[y] = x;
      return true;
   }
   int same(int x, int y) { return find(x) == find(y); }
   int size(int x) { return (-data[find(x)]); }
   void undo() {
      data[history.top().first] = history.top().second;
      history.pop();
      data[history.top().first] = history.top().second;
      history.pop();
   }
   int time() { return int(history.size() >> 1); }
   void snapshot() { inner_snap = time(); }
   void rollback(int t = -1) {
      if(t == -1) t = inner_snap;
      while(t < time()) undo();
   }
};
```

UnionFind.hpp

md5: 8f4a82

```
struct UnionFind {
   vector<int> par, siz;
   UnionFind(int n) : par(n, -1), siz(n, 1) {}
   int root(int x) {
      if(par[x] == -1) return x;
      else return par[x] = root(par[x]);
   }
   bool issame(int x, int y) { return root(x) == root(y); }
   bool unite(int x, int y) {
      x = root(x), y = root(y);
      if(x == y) return false;
      if(siz[x] < siz[y]) swap(x, y);
      par[y] = x;
      siz[x] += siz[y];
      return true;
   }
   int size(int x) { return siz[root(x)]; }
};
```

fenwick_tree.hpp

md5: 60d4dd

```
struct fenwick_tree {
   vector<ll> a;
   fenwick_tree(ll n) : a(n + 1) {}
   void add(ll i, ll x) {  // A[i] += x
      i++;
      while(i < sz(a)) {
         a[i] += x;
         i += i & -i;
      }
   }
   ll sum(ll r) {
      ll s = 0;
      while(r) {
         s += a[r];
         r -= r & -r;
      }
      return s;
   }
   ll sum(ll l, ll r) {  // sum of A[l, r)
      return sum(r) - sum(l);
   }
   ll lower_bound(ll x) {  // sum(i+1) >= x となる最小の i。無けれ
ば n。A[i] >= 0 が前提
      ll i = 0, n = sz(a) - 1, k = 1;
      while(k * 2 <= n) k *= 2;
      for(; k; k /= 2)
         if(i + k <= n && a[i + k] < x) i += k, x -= a[i];
      return i;
   }
};
```

lazy_segtree.hpp

md5: 4a7b33

```
// https://github.com/YNUCPC/ynu-icpc-
library/blob/main/src/datastructure/lazysegtree.hpp
template<class S, S (*op)(S, S), S (*e)(), class F, S
(*mapping)(F, S), F (*composition)(F, F), F (*id)()>
struct lazy_segtree {
   private:
   int _n, size, log;
   vector<S> dat;
   vector<F> lz;
   void update(int k) { dat[k] = op(dat[2 * k], dat[2 * k +
1]); }
   void all_apply(int k, F f) {
      dat[k] = mapping(f, dat[k]);
      if(k < size) lz[k] = composition(f, lz[k]);
   }
   void push(int k) {
      all_apply(2 * k, lz[k]);
      all_apply(2 * k + 1, lz[k]);
      lz[k] = id();
   }
   int lower_bits(int x, int k) { return x & ((1 << k) - 1); }

   public:
   lazy_segtree() : lazy_segtree(0) {}
   lazy_segtree(int n) : lazy_segtree(vector<S>(n, e())) {}
```

```
    lazy_segtree(const vector<S>& v) : _n(int(v.size())) {
        log = 0;
        while((1 << log) < _n) log++;
        size = 1 << log;
        dat = vector<S>(2 * size, e());
        lz = vector<F>(size, id());
        for(int i = 0; i < _n; i++) dat[size + i] = v[i];
        for(int i = size - 1; i >= 1; i--) update(i);
    }
    // a[p] = x
    void set(int p, S x) {
        p += size;
        for(int i = log; i >= 1; i--) push(p >> i);
        dat[p] = x;
        for(int i = 1; i <= log; i++) update(p >> i);
    }
    // return a[p]
    S get(int p) {
        p += size;
        for(int i = log; i >= 1; i--) push(p >> i);
        return dat[p];
    }
    // return op(a[l], ..., a[r-1])
    S prod(int l, int r) {
        if(l == r) return e();
        l += size;
        r += size;
        for(int i = log; i >= 1; i--) {
            if(lower_bits(l, i) > 0) push(l >> i);
            if(lower_bits(r, i) > 0) push((r - 1) >> i);
        }
        S sml = e(), smr = e();
        while(l < r) {
            if(l & 1) sml = op(sml, dat[l++]);
            if(r & 1) smr = op(dat[--r], smr);
            l >>= 1;
            r >>= 1;
        }
        return op(sml, smr);
    }
    S all_prod() { return dat[1]; }
    // a[p] = f(a[p])
    void apply(int p, F f) {
        p += size;
        for(int i = log; i >= 1; i--) push(p >> i);
        dat[p] = mapping(f, dat[p]);
        for(int i = 1; i <= log; i++) update(p >> i);
    }
    // i = l...r-1 について a[i] = f(a[i])
    void apply(int l, int r, F f) {
        if(l == r) return;
        l += size;
        r += size;
        for(int i = log; i >= 1; i--) {
            if(lower_bits(l, i) > 0) push(l >> i);
            if(lower_bits(r, i) > 0) push((r - 1) >> i);
        }
        int l2 = l, r2 = r;
        while(l < r) {
            if(l & 1) all_apply(l++, f);
            if(r & 1) all_apply(--r, f);
            l >>= 1;
            r >>= 1;
        }
        l = l2;
        r = r2;
        for(int i = 1; i <= log; i++) {
            if(lower_bits(l, i) > 0) update(l >> i);
            if(lower_bits(r, i) > 0) update((r - 1) >> i);
        }
    }
    // SegmentTree上の二分探索（必要な場合）
    // return r, f(op(a[l], ..., a[r-1])) == true
    template<bool (*g)(S)> int max_right(int l) {
        return max_right(l, [](S x) { return g(x); });
    }
    template<class G> int max_right(int l, G g) {
        assert(g(e()));
        if(l == _n) return _n;
        l += size;
        for(int i = log; i >= 1; i--) push(l >> i);
        S sm = e();
        do {
            while(l % 2 == 0) l >>= 1;
            if(!g(op(sm, dat[l]))) {
                while(l < size) {
                    push(l);
                    l = (2 * l);
                    if(g(op(sm, dat[l]))) {
                        sm = op(sm, dat[l]);
                        l++;
                    }
                }
                return l - size;
            }
            sm = op(sm, dat[l]);
            l++;
        } while((l & -l) != l);
        return _n;
    }
    // return l, f(op(a[l], ..., a[r-1])) == true
    template<bool (*g)(S)> int min_left(int r) {
        return min_left(r, [](S x) { return g(x); });
    }
    template<class G> int min_left(int r, G g) {
        assert(g(e()));
        if(r == 0) return 0;
        r += size;
        for(int i = log; i >= 1; i--) push((r - 1) >> i);
        S sm = e();
        do {
            r--;
            while(r > 1 && (r % 2)) r >>= 1;
            if(!g(op(dat[r], sm))) {
                while(r < size) {
                    push(r);
                    r = (2 * r + 1);
                    if(g(op(dat[r], sm))) {
                        sm = op(dat[r], sm);
                        r--;
                    }
                }
                return r + 1 - size;
            }
            sm = op(dat[r], sm);
        } while((r & -r) != r);
        return 0;
    }
};
```

segtree.hpp

md5: 78c480

```
// https://github.com/YNUCPC/ynu-icpc-library/blob/main/src/datastructure/segtree.hpp
template<class S, S (*op)(S, S), S (*e)()> struct SegmentTree {
    private:
    int _n, size, log;
    vector<S> dat;
    void update(int k) { dat[k] = op(dat[2 * k], dat[2 * k + 1]); }

    public:
    SegmentTree() : SegmentTree(0) {}
    SegmentTree(int n) : SegmentTree(vector<S>(n, e())) {}
    SegmentTree(const vector<S>& v) : _n(int(v.size())) {
        log = 0;
        while((1 << log) < _n) log++;
        size = 1 << log;
        dat = vector<S>(2 * size, e());
        for(int i = 0; i < _n; i++) dat[size + i] = v[i];
        for(int i = size - 1; i >= 1; i--) { update(i); }
    }
    // a[p] = x
    void set(int p, S x) {
        p += size;
        dat[p] = x;
        for(int i = 1; i <= log; i++) update(p >> i);
    }
    // return a[p]
    S get(int p) const { return dat[p + size]; }
    // return op(a[l], ..., a[r-1])
    S prod(int l, int r) const {
        S sml = e(), smr = e();
        l += size;
        r += size;
        while(l < r) {
            if(l & 1) sml = op(sml, dat[l++]);
            if(r & 1) smr = op(dat[--r], smr);
            l >>= 1;
            r >>= 1;
        }
```

```
      return op(sml, smr);
   }
   S all_prod() const { return dat[1]; }

   // SegmentTree上の二分探索 (必要な場合)
   // return r, f(op(a[l], ..., a[r-1])) == true
   template<bool (*f)(S)> int max_right(int l) const {
      return max_right(l, [](S x) { return f(x); });
   }
   template<class F> int max_right(int l, F f) const {
      assert(f(e()));
      if(l == _n) return _n;
      l += size;
      S sm = e();
      do {
         while(l % 2 == 0) l >>= 1;
         if(!f(op(sm, dat[l]))) {
            while(l < size) {
               l = (2 * l);
               if(f(op(sm, dat[l]))) {
                  sm = op(sm, dat[l]);
                  l++;
               }
            }
            return l - size;
         }
         sm = op(sm, dat[l]);
         l++;
      } while((l & -l) != l);
      return _n;
   }
   // return l, f(op(a[l], ..., a[r-1])) == true
   template<bool (*f)(S)> int min_left(int r) const {
      return min_left(r, [](S x) { return f(x); });
   }
   template<class F> int min_left(int r, F f) const {
      assert(f(e()));
      if(r == 0) return 0;
      r += size;
      S sm = e();
      do {
         r--;
         while(r > 1 && (r % 2)) r >>= 1;
         if(!f(op(dat[r], sm))) {
            while(r < size) {
               r = (2 * r + 1);
               if(f(op(dat[r], sm))) {
                  sm = op(dat[r], sm);
                  r--;
               }
            }
            return r + 1 - size;
         }
         sm = op(dat[r], sm);
      } while((r & -r) != r);
      return 0;
   }
};
```

math

BinaryGCD.hpp

md5: f3ab31

```
u64 ctz(u64 x) { return countr_zero(x); }
u64 binary_gcd(u64 x, u64 y) {
   if(!x || !y) return x | y;
   u64 n = ctz(x), m = ctz(y);
   x >>= n, y >>= m;
   while(x != y) {
      if(x > y) x = (x - y) >> ctz(x - y);
      else y = (y - x) >> ctz(y - x);
   }
   return x << min(n, m);
}
```

EulerPhi.hpp

md5: 2443cb

```
int64_t euler_phi(int64_t n) {
   int64_t ret = n;
   for(int64_t i = 2; i * i <= n; i++) {
      if(n % i == 0) {
         ret -= ret / i;
         while(n % i == 0) n /= i;
      }
   }
   if(n > 1) ret -= ret / n;
   return ret;
}
```

ExtGCD.hpp

md5: c3fa9b

```
// returns gcd(a, b) and assign x, y to integers
// s.t. ax + by = gcd(a, b) and |x| + |y| is minimized
ll extgcd(ll a, ll b, ll& x, ll& y) {
   // assert(a >= 0 && b >= 0);
   if(!b) return x = 1, y = 0, a;
   ll d = extgcd(b, a % b, y, x);
   y -= a / b * x;
   return d;
}
```

Matrix.hpp

md5: fe12c1

```
// https://youtu.be/ylWYSurx10A?t=2352
template<typename T> struct Matrix {
   int h, w;
   vector<vector<T>> d;
   Matrix() {}
   Matrix(int h, int w, T val = 0) : h(h), w(w), d(h, vector<T>
(w, val)) {}
   Matrix& unit() {
      // assert(h == w);
      rep(i, h) d[i][i] = 1;
      return *this;
   }
   const vector<T>& operator[](int i) const { return d[i]; }
   vector<T>& operator[](int i) { return d[i]; }
   Matrix operator*(const Matrix& a) const {
      //   assert(w == a.h);
      Matrix r(h, a.w);
      rep(i, h) rep(k, w) rep(j, a.w) { r[i][j] += d[i][k] *
a[k][j]; }
      return r;
   }
   Matrix pow(long long t) const {
      //  assert(h == w);
      if(!t) return Matrix(h, h).unit();
      if(t == 1) return *this;
      Matrix r = pow(t >> 1);
      r = r * r;
      if(t & 1) r = r * (*this);
      return r;
   }
   // https://youtu.be/-j02o6__jgs?t=11273
   /* mint only
   mint det() {
     assert(h == w);
     mint res = 1;
     rep(k,h) {
       for (int i = k; i < h; ++i) {
         if (d[i][k] == 0) continue;
         if (i != k) {
           swap(d[i],d[k]);
           res = -res;
         }
       }
       if (d[k][k] == 0) return 0;
       res *= d[k][k];
       mint inv = mint(1)/d[k][k];
       rep(j,h) d[k][j] *= inv;
       for (int i = k+1; i < h; ++i) {
         mint c = d[i][k];
         for (int j = k; j < h; ++j) d[i][j] -= d[k][j]*c;
       }
     }
     return res;
   }
   //*/
};
```

floor_sum.hpp

md5: 930ca0

```
// x_i=floor((a*i+b)/c), i=0,1,..n-1
// a,c>0, b>=0
ll floor_sum(ll n, ll a, ll b, ll c) {
   if(n == 0) return 0;
   ll res = 0;
   res += n * (n - 1) / 2 * (a / c);
   a %= c;
```

```
   res += n * (b / c);
   b %= c;
   if(a == 0) return res;
   ll top = (a * (n - 1) + b) / c;
   res += top * n;
   ll h = (b + 1 + c - 1) / c;
   if(h <= top) res -= floor_sum(top - h + 1, c, c * h - (b +
1), a) + top - h + 1;
   return res;
}
```

gensikon.hpp

md5: 19c243

```
template<class Int = long long> Int
find_smallest_primitive_root(Int p) {
   std::vector<Int> fac;
   Int v = p - 1;
   for(Int pp = 2; pp * pp <= v; pp++) {  // prime
factorization of (p - 1)
      int e = 0;
      while(v % pp == 0) e++, v /= pp;
      if(e) fac.push_back(pp);
   }
   if(v > 1) fac.push_back(v);

   for(Int g = 1; g < p; g++) {
      if(pow_mod(g, p - 1, p) != 1) return -1;
      bool ok = true;
      for(auto pp : fac) {
         if(pow_mod(g, (p - 1) / pp, p) == 1) {
            ok = false;
            break;
         }
      }
      if(ok) return g;
   }
   return -1;
}
```

lagrange.hpp

md5: c96283

```
#define FOR(i, a, n) for(ll i = (ll)a; i < (ll)n; ++i)
#define REP(i, n) FOR(i, 0, n)
#define ALL(x) x.begin(), x.end()

// x座標が相異なるn+1点(x_i,y_i)を通るn次以下の多項式f(x)を返す
// O(n^2)
vector<mint> lagrange_interpolation(vector<mint> x,
vector<mint> y) {
   const ll n = x.size() - 1;
   REP(i, n + 1) {
      mint t = 1;
      REP(j, n + 1) if(i != j) t *= x[i] - x[j];
      y[i] /= t;
   }
   ll cur = 0, nxt = 1;
   vector<vector<mint>> dp(2, vector<mint>(n + 2));
   dp[0][0] = -1 * x[0], dp[0][1] = 1;
   FOR(i, 1, n + 1) {
      REP(j, n + 2) {
         dp[nxt][j] = dp[cur][j] * -1 * x[i];
         if(j >= 1) dp[nxt][j] += dp[cur][j - 1];
      }
      swap(nxt, cur);
   }
   vector<mint> xinv(n + 1);
   REP(i, n + 1) xinv[i] = x[i].inv();
   vector<mint> ret(n + 1);  // f(x)
   REP(i, n + 1) {
      if(y[i] == 0) continue;
      // 0割り対策の場合分け
      if(x[i] == 0) {
         REP(j, n + 1) ret[j] += dp[cur][j + 1] * y[i];
      } else {
         ret[0] -= dp[cur][0] * xinv[i] * y[i];
         mint pre = -1 * dp[cur][0] * xinv[i];
         FOR(j, 1, n + 1) {
            ret[j] -= (dp[cur][j] - pre) * xinv[i] * y[i];
            pre = -1 * (dp[cur][j] - pre) * xinv[i];
         }
      }
   }
   return ret;
}
```

```
mint lagrange_interpolationT(vector<mint> x, vector<mint> y,
mint T) {
   const ll n = x.size() - 1;
   mint ret = 0;
   REP(i, n + 1) {
      mint t = 1;
      REP(j, n + 1) {
         if(i == j) continue;
         t *= T - x[j];
         t /= x[i] - x[j];
      }
      ret += t * y[i];
   }
   return ret;
}
```

linear_congruence.hpp

md5: c6a31e

```
ll extgcd(ll a, ll b, ll& x, ll& y) {
   ll d = a;
   if(b != 0) {
      d = extgcd(b, a % b, y, x);
      y -= (a / b) * x;
   } else {
      x = 1;
      y = 0;
   }
   return d;
}
ll mod_inverse(ll a, ll m) {
   ll x, y;
   extgcd(a, m, x, y);
   return (m + x % m) % m;
}

P linear_congruence(vll& A, vll& B, vll& M) {
   ll x = 0, m = 1;
   rep(i, A.size()) {
      ll a = A[i] * m, b = B[i] - A[i] * x, d = gcd(M[i], a);
      if(b % d) return P(0, -1);
      ll t = (b / d) * mod_inverse(a / d, M[i] / d) % (M[i] /
d);
      x = x + m * t;
      m *= M[i] / d;
   }
   return P((x % m + m) % m, m);
}
```

mathtemplate.hpp

md5: 8743ef

```
ll pow_pow(ll x, ll n, ll mod) {
   if(n == 0) return 1;
   x %= mod;
   ll res = pow_pow(x * x % mod, n / 2, mod);
   if(n & 1) res = res * x % mod;
   return res;
}
ll gcdll(ll x, ll y) {
   if(y == 0) return x;
   return gcdll(y, x % y);
}
ll INFF = 1000100100100100100;
ll lcmll(ll x, ll y) {
   ll X = (x / gcdll(x, y)), Y = y;
   if(X <= INFF / Y) return X * Y;
   else return INFF;
}
```

xorConvolution.hpp

md5: b6be6f

```
//
https://github.com/KentaroMatsushita/icpc_library/blob/main/src/m
convolution.hpp
// sizeは2べき
int my_lg(int n) { return 31 - __builtin_clz(n); }
#define si(a) (int)(size(a))
#define fore(e, v) for(auto&& e : v)
template<typename T> void fwt(vector<T>& f, bool inv = false) {
   int n = si(f), m = my_lg(n);
   rep(i, m) {
      rep(b, n) {
         if(~b >> i & 1) {
            T x = f[b], y = f[b ^ 1 << i];
            f[b] = x + y, f[b ^ 1 << i] = x - y;
         }
```

```
      }
   }
   if(inv) {
      T iz = T(1) / T(si(f));
      fore(e, f) e *= iz;
   }
}
/*
void solve(int test){
    vm a={1,1,1};
    vm b={1,1,1};
    fwt<mint>(a);
    fwt<mint>(b);
    vm c=a;
    rep(i,c.size())c[i]=a[i]*b[i];
    fwt<mint>(c,true);
}
*/
```

modint

BarrettReduction.hpp

md5: 2ca7f3

```
// using u64 = uint64_t;
struct Barrett {  // mod < 2^32
   u64 m, im;
   Barrett(u64 mod) : m(mod), im(-1ULL / m + 1) {}
   // input: a * b < 2^64, output: a * b % mod
   u64 mul(u64 a, u64 b) const {
      a *= b;
      u64 x = ((__uint128_t)a * im) >> 64;
      a -= x * m;
      if((ll)a < 0) a += m;
      return a;
   }
};
```

combination.hpp

md5: 7f04ef

```
constexpr int MAX = 10;
ll fact[MAX], finv[MAX], invv[MAX];
void initcomb() {
   fact[0] = fact[1] = 1;
   finv[0] = finv[1] = 1;
   invv[1] = 1;
   for(int i = 2; i < MAX; i++) {
      fact[i] = fact[i - 1] * i % mod;
      invv[i] = mod - invv[mod % i] * (mod / i) % mod;
      finv[i] = finv[i - 1] * invv[i] % mod;
   }
}
ll comb(ll n, ll k) {
   if(n < k) return 0;
   if(n < 0 || k < 0) return 0;
   return fact[n] * (finv[k] * finv[n - k] % mod) % mod;
}
```

convolution.hpp

md5: 673c1c

```
using vm = vector<mint>;
int my_lg(int n) { return 31 - __builtin_clz(n); }

//
https://github.com/KentaroMatsushita/icpc_library/blob/main/src/FPS/FFT.hpp
#include <bit>
#define si(a) (int)(size(a))
#define fore(e, v) for(auto&& e : v)
#define eb emplace_back
mint g = 3;  // 原始根
void fft(vm& a, bool inv = false) {
   int n = si(a), s = my_lg(n);
   static vm z, iz;
   while(si(z) <= s) {
      z.eb(g.pow(mint(-1).x / (1 << si(z))));
      iz.eb(z.back().inv());
   }
   vm b(n);
   repi(i, 1, s + 1) {
      int w = 1 << s - i;
      mint base = inv ? iz[i] : z[i], now = 1;
      for(int y = 0; y < n / 2; y += w) {
         rep(x, w) {
            auto l = a[y << 1 | x], r = now * a[y << 1 | x |
w];
            b[y | x] = l + r, b[y | x | n >> 1] = l - r;
         }
         now *= base;
      }
      swap(a, b);
   }
}

vm mul(vm a, vm b) {
   int n = si(a), m = si(b);
   if(!n or !m) return {};
   if(min(n, m) <= 30) {
      vm ans(n + m - 1);
      rep(i, n) rep(j, m) ans[i + j] += a[i] * b[j];
      return ans;
   }
   int N = n + m - 1;
   int z = bit_ceil(unsigned(N));
   a.resize(z), b.resize(z);
   fft(a), fft(b);
   rep(i, z) a[i] *= b[i];
   fft(a, true);
   a.resize(n + m - 1);
   mint iz = mint(z).inv();
   fore(e, a) e *= iz;
   return a;
}
```

modint.hpp

md5: 3db9f2

```
constexpr int mod = 998244353;
struct mint {
   int x;
   mint(ll x_ = 0) : x(x_ % mod) {
      if(x < 0) x += mod;
   }
   mint operator-() {
      auto res = *this;
      res.x = (x ? mod - x : 0);
      return res;
   }
   mint& operator+=(mint r) {
      if((x += r.x) >= mod) x -= mod;
      return *this;
   }
   mint& operator-=(mint r) {
      if((x -= r.x) < 0) x += mod;
      return *this;
   }
   mint& operator*=(mint r) {
      x = 1LL * x * r.x % mod;
      return *this;
   }
   mint& operator/=(mint r) { return *this *= r.inv(); }
   friend mint operator+(mint a, mint b) { return a += b; }
   friend mint operator-(mint a, mint b) { return a -= b; }
   friend mint operator*(mint a, mint b) { return a *= b; }
   friend mint operator/(mint a, mint b) { return a /= b; }
   mint inv() const { return pow(mod - 2); }
   mint pow(ll b) const {
      mint a = *this, c = 1;
      while(b) {
         if(b & 1) c *= a;
         a *= a;
         b >>= 1;
      }
      return c;
   }
};
using vm = vector<mint>;
```

extra

modint_fast.hpp

md5: 4b6639

```
const uint32_t mod = 998244353;
struct mm {
   uint32_t x;
   mm() : x(0) {}
   template<class T> mm(T x_) : x(x_ % mod) {
      if(x >= mod) x += mod;
   }
   friend mm operator+(mm a, mm b) {
      a.x += b.x;
```

```
      if(a.x >= mod) a.x -= mod;
      return a;
   }
   friend mm operator-(mm a, mm b) {
      a.x -= b.x;
      if(a.x >= mod) a.x += mod;
      return a;
   }
   friend mm operator*(mm a, mm b) { return (uint64_t)a.x *
b.x; }
   friend mm operator/(mm a, mm b) { return a * b.inv(); }
   friend mm& operator+=(mm& a, mm b) { return a = a + b; }
   friend mm& operator-=(mm& a, mm b) { return a = a - b; }
   friend mm& operator*=(mm& a, mm b) { return a = a * b; }
   friend mm& operator/=(mm& a, mm b) { return a = a * b.inv();
}
   mm inv() const { return pow(mod - 2); }
   mm pow(ll b) const {
      mm a = *this, c = 1;
      while(b) {
         if(b & 1) c *= a;
         a *= a;
         b >>= 1;
      }
      return c;
   }
};
```

FPS

FFT.hpp

md5: 3138c7

```
// {998244353, 3}, {1811939329, 13}, {2013265921, 31}
mm g = 3;  // 原始根
void fft(vector<mm>& a) {
   ll n = sz(a), lg = __lg(n);
   assert((1 << lg) == n);
   vector<mm> b(n);
   rep(l, 1, lg + 1) {
      ll w = n >> l;
      mm s = 1, r = g.pow(mod >> l);
      for(ll u = 0; u < n / 2; u += w) {
         rep(d, 0, w) {
            mm x = a[u << 1 | d], y = a[u << 1 | w | d] * s;
            b[u | d] = x + y;
            b[n >> 1 | u | d] = x - y;
         }
         s *= r;
      }
      swap(a, b);
   }
}
vector<mm> conv(vector<mm> a, vector<mm> b) {
   if(a.empty() || b.empty()) return {};
   size_t s = sz(a) + sz(b) - 1, n = bit_ceil(s);
   // if(min(sz(a), sz(b)) <= 60) 愚直に掛け算
   a.resize(n);
   b.resize(n);
   fft(a);
   fft(b);
   mm inv = mm(n).inv();
   rep(i, 0, n) a[i] *= b[i] * inv;
   reverse(1 + all(a));
   fft(a);
   a.resize(s);
   return a;
}
```

FFT_fast.hpp

md5: c8c567

```
// modint を u32 にして加減算を真面目にやると速い
mm g = 3;  // 原始根
void fft(vector<mm>& a) {
   ll n = sz(a), lg = __lg(n);
   static auto z = [] {
      vector<mm> z(30);
      mm s = 1;
      rep(i, 2, 32) {
         z[i - 2] = s * g.pow(mod >> i);
         s *= g.inv().pow(mod >> i);
      }
      return z;
   }();
   rep(l, 0, lg) {
      ll w = 1 << (lg - l - 1);
      mm s = 1;
      rep(k, 0, 1 << l) {
         ll o = k << (lg - l);
         rep(i, o, o + w) {
            mm x = a[i], y = a[i + w] * s;
            a[i] = x + y;
            a[i + w] = x - y;
         }
         s *= z[countr_zero<uint64_t>(~k)];
      }
   }
}
// コピペ
void ifft(vector<mm>& a) {
   ll n = sz(a), lg = __lg(n);
   static auto z = [] {
      vector<mm> z(30);
      mm s = 1;
      rep(i, 2, 32) {  // g を逆数に
         z[i - 2] = s * g.inv().pow(mod >> i);
         s *= g.pow(mod >> i);
      }
      return z;
   }();
   for(ll l = lg; l--;) {  // 逆順に
      ll w = 1 << (lg - l - 1);
      mm s = 1;
      rep(k, 0, 1 << l) {
         ll o = k << (lg - l);
         rep(i, o, o + w) {
            mm x = a[i], y = a[i + w];  // *s を下に移動
            a[i] = x + y;
            a[i + w] = (x - y) * s;
         }
         s *= z[countr_zero<uint64_t>(~k)];
      }
   }
}
vector<mm> conv(vector<mm> a, vector<mm> b) {
   if(a.empty() || b.empty()) return {};
   size_t s = sz(a) + sz(b) - 1, n = bit_ceil(s);
   // if(min(sz(a), sz(b)) <= 60) 愚直に掛け算
   a.resize(n);
   b.resize(n);
   fft(a);
   fft(b);
   mm inv = mm(n).inv();
   rep(i, 0, n) a[i] *= b[i] * inv;
   ifft(a);
   a.resize(s);
   return a;
}
```

graph

CentroidDecomposition.cpp

md5: dae9cb

```
template< typename G >
struct CentroidDecomposition {
  const G &g;
  vector< int > sub;
  vector< vector< int > > belong;
  vector< bool > v;

  CentroidDecomposition(const G &g) : g(g), sub(g.size()),
v(g.size()), belong(g.size()) {}

  inline int build_dfs(int idx, int par) {
    sub[idx] = 1;
    for(auto &to : g[idx]) {
      if(to == par || v[to]) continue;
      sub[idx] += build_dfs(to, idx);
    }
    return sub[idx];
  }

  inline int search_centroid(int idx, int par, const int mid) {
    for(auto &to : g[idx]) {
      if(to == par || v[to]) continue;
      if(sub[to] > mid) return search_centroid(to, idx, mid);
    }
    return idx;
  }
```

```
  inline void belong_dfs(int idx, int par, int centroid) {
    belong[idx].emplace_back(centroid);
    for(auto &to : g[idx]) {
      if(to == par || v[to]) continue;
      belong_dfs(to, idx, centroid);
    }
  }

  inline int build(G &t, int idx) {
    int centroid = search_centroid(idx, -1, build_dfs(idx, -1)
/ 2);
    v[centroid] = true;
    belong_dfs(centroid, -1, centroid);
    for(auto &to : g[centroid]) {
      if(!v[to]) t[centroid].emplace_back(build(t, to));
    }
    v[centroid] = false;
    return centroid;
  }

  inline int build(G &t) {
    t.resize(g.size());
    return build(t, 0);
  }
};

vll anss;
void solve(int test){
    int n; cin >>  n;
    vvi g(n);
    rep(i,n-1){
        int a,b; cin >> a >> b;a--;b--;
        g[a].push_back(b);
        g[b].push_back(a);
    }
    CentroidDecomposition<vvi> cd(g);
    vvi tr;
    int rt=cd.build(tr);
    rep(i,n){
        cout << i << " ";
        for(auto u:tr[i])cout << u << " ";
        cout << endl;
    }
}
```

TreeHash.hpp

md5: c07d91

```
using ull = unsigned long long;
struct TreeHash {
   const vvi& g;
   vector<ull> h, rnd;
   vi ht;
   mt19937_64
mt{(ull)chrono::steady_clock::now().time_since_epoch().count()};
   TreeHash(const vvi& g) : g(g), h(sz(g)), ht(sz(g)) {}
   ull operator()(vi& ng, int r) {
      int sv = ng[r];
      ng[r] = 1, dfs(ng, r), ng[r] = sv;
      return h[r];
   }
   void dfs(vi& ng, int v) {
      h[v] = 1, ht[v] = 0;
      for(int u : g[v]) if(!ng[u]) {
         ng[u] = 1, dfs(ng, u), ng[u] = 0;
         while(sz(rnd) <= ht[u]) rnd.push_back(mt());
         h[v] *= (h[u] + rnd[ht[u]]) | 1;
         chmax(ht[v], ht[u] + 1);
      }
   }
};
```

Trie.hpp

md5: 63ded2

```
struct Trie {
   int cz;
   vvi g;
   int size;
   vll dp;
   Trie(int cz = 26) : cz(cz), g(1, vi(cz)), size(1), dp(1) {}
   void add(vi& s) {
      int si = s.size();
      int id = 0;
      rep(j, si) {
         if(g[id][s[j]]) {
            id = g[id][s[j]];
         } else {
            g[id][s[j]] = size;
            id = size;
            size++;
            g.push_back(vi(cz));
            dp.push_back(0);
         }
         dp[id]++;
      }
   }
};
```

Trie_yokohama.hpp

md5: 73d6b5

```
#pragma once
#include <bits/stdc++.h>
using namespace std;

struct Trie {
   static constexpr int C_SIZE = 26;    // C_SIZE  : 文字の種類数
   static constexpr int C_BEGIN = 'a';  // C_BEGIN : 開始文字
   static constexpr int ROOT = 0;
   struct Node {
      array<int, C_SIZE> to = {};  // 子ノードの番号，存在しなけれ
ば-1
      vector<int> ids;             // そのノードが終端である文字列の
IDリスト
      Node() { to.fill(-1); }
   };
   vector<Node> nodes;
   int cnt = 0;  // 追加した文字列の個数
   Trie() : nodes(1) {}
   // nodes[idx]から文字cで遷移したときの頂点のindex
   int next(int idx, char c) { return nodes[idx].to[c -
C_BEGIN]; }
   // 文字列の追加
   int insert(const string& s) {
      int now = ROOT;
      for(char c : s) {
         int k = c - C_BEGIN;
         if(nodes[now].to[k] == -1) {
            nodes[now].to[k] = nodes.size();
            nodes.push_back(Node());
         }
         now = nodes[now].to[k];
      }
      nodes[now].ids.push_back(cnt++);
      return now;
   }
   // 文字列に対応するnodeのindexを検索，存在しなければ-1
   int find(const string& s) {
      int now = ROOT;
      for(char c : s) {
         now = next(now, c);
         if(now == -1) return -1;
      }
      return now;
   }
};
```

dijkstra.hpp

md5: e9adaf

```
auto f = [&](int s) {
   vll dist(n, inf);
   priority_queue<P, vector<P>, greater<P>> q;
   dist[s] = 0;
   q.push(P(dist[s], s));
   while(!q.empty()) {
      int v = q.top().second;
      ll d = q.top().first;
      q.pop();
      if(dist[v] < d) continue;
      for(auto u : g[v]) {
         if(chmin(dist[u.to], dist[v] + u.cost)) {
q.push(P(dist[u.to], u.to)); }
      }
   }
};
```

edge.hpp

md5: 111ed8

```
struct edge {
   int to;
```

```
   ll cost;
   int id;
   edge(int to, ll cost, int id) : to(to), cost(cost), id(id)
{}
};
```

eulertrail.hpp

md5: 83e15f

```
// https://kokiymgch.hatenablog.com/entry/2017/12/07/193238
// gが隣接リスト
// gを破壊する
vector<int> EulerianTrail(const int s, vector<vector<int>>& g,
const bool directed) {
   function<void(int, vector<int>&)> dfs = [&](int u,
vector<int>& trail) {
      while(!g[u].empty()) {
         int v = g[u].back();
         g[u].pop_back();
         if(!directed) {
            for(int i = 0; i < g[v].size(); i++) {
               if(g[v][i] == u) {
                  g[v].erase(g[v].begin() + i);
                  break;
               }
            }
         }
         dfs(v, trail);
      }
      trail.push_back(u);
   };
   vector<int> trail;
   dfs(s, trail);
   reverse(trail.begin(), trail.end());
   return trail;
}
```

gyouretsukiteiri.hpp

md5: 5ce1b7

```
template<typename T> struct Matrix {
   int h, w;
   vector<vector<T>> d;
   Matrix() {}
   Matrix(int h, int w, T val = 0) : h(h), w(w), d(h, vector<T>
(w, val)) {}
   Matrix& unit() {
      assert(h == w);
      rep(i, h) d[i][i] = 1;
      return *this;
   }
   const vector<T>& operator[](int i) const { return d[i]; }
   vector<T>& operator[](int i) { return d[i]; }
   Matrix operator*(const Matrix& a) const {
      assert(w == a.h);
      Matrix r(h, a.w);
      rep(i, h) rep(k, w) rep(j, a.w) { r[i][j] += d[i][k] *
a[k][j]; }
      return r;
   }
   Matrix pow(long long t) const {
      assert(h == w);
      if(!t) return Matrix(h, h).unit();
      if(t == 1) return *this;
      Matrix r = pow(t >> 1);
      r = r * r;
      if(t & 1) r = r * (*this);
      return r;
   }
   mint det() {
      assert(h == w);
      mint res = 1;
      rep(k, h) {
         for(int i = k; i < h; ++i) {
            if(d[i][k] == 0) continue;
            if(i != k) {
               swap(d[i], d[k]);
               res = -res;
            }
         }
         if(d[k][k] == 0) return 0;
         res *= d[k][k];
         mint inv = mint(1) / d[k][k];
         rep(j, h) d[k][j] *= inv;
         for(int i = k + 1; i < h; ++i) {
            mint c = d[i][k];
            for(int j = k; j < h; ++j) d[i][j] -= d[k][j] * c;
         }
      }
      return res;
   }
};
l[i][j] = deg(i) if(i == j) iとjの辺の本数
          * (-1) if(i != j)

              これの余因子が全域木々の個数(-1)
          * *(0 + 0) と0行0列を除いた行列式をかけたものでok
```

hld.hpp

md5: 1f23da

```
//
https://github.com/KentaroMatsushita/icpc_library/blob/main/src/t
#define si(a) (int)(size(a))
#define fore(e, v) for(auto&& e : v)
template<typename G> struct HLD {
   int n;
   G& g;
   vector<int> sub, in, out, head, rev, par, d;
   HLD(G& g) : n(si(g)), g(g), sub(n), in(n), out(n), head(n),
rev(n), par(n), d(n) {}
   void dfs1(int x, int p) {
      par[x] = p;
      sub[x] = 1;
      if(g[x].size() and g[x][0] == p) swap(g[x][0],
g[x].back());
      fore(e, g[x]) {
         if(e == p) continue;
         d[e] = d[x] + 1;
         dfs1(e, x);
         sub[x] += sub[e];
         if(sub[g[x][0]] < sub[e]) swap(g[x][0], e);
      }
   }
   void dfs2(int x, int p, int& t) {
      in[x] = t++;
      rev[in[x]] = x;
      fore(e, g[x]) {
         if(e == p) continue;
         head[e] = (g[x][0] == e ? head[x] : e);
         dfs2(e, x, t);
      }
      out[x] = t;
   }
   void build() {
      int t = 0;
      head[0] = 0;
      dfs1(0, -1);
      dfs2(0, -1, t);
   }
   int la(int v, int k) {
      while(1) {
         int u = head[v];
         if(in[v] - k >= in[u]) return rev[in[v] - k];
         k -= in[v] - in[u] + 1;
         v = par[u];
      }
   }
   int lca(int u, int v) {
      for(;; v = par[head[v]]) {
         if(in[u] > in[v]) swap(u, v);
         if(head[u] == head[v]) return u;
      }
   }
   template<typename T, typename Q, typename F>
   T query(int u, int v, const T& e, const Q& q, const F& f,
bool edge = false) {
      T l = e, r = e;
      for(;; v = par[head[v]]) {
         if(in[u] > in[v]) swap(u, v), swap(l, r);
         if(head[u] == head[v]) break;
         l = f(q(in[head[v]], in[v] + 1), l);
      }
      return f(f(q(in[u] + edge, in[v] + 1), l), r);
   }
   int dist(int u, int v) { return d[u] + d[v] - 2 * d[lca(u,
v)]; }
   int jump(int s, int t, int i) {
      if(!i) return s;
      int l = lca(s, t);
      int dst = d[s] + d[t] - d[l] * 2;
      if(dst < i) return -1;
      if(d[s] - d[l] >= i) return la(s, i);
```

```
      i -= d[s] - d[l];
      return la(t, d[t] - d[l] - i);
   }
};
// 以下、使い方
using S = ll;
S op(S a, S b) { return a + b; }
S e() { return 0ll; }
void solve(int test) {
   int n;
   cin >> n;
   int m = n - 1;
   vi u(m), v(m), w(m);
   vvi g(n);
   rep(i, m) {
      cin >> u[i] >> v[i] >> w[i];
      u[i]--, v[i]--;
      g[u[i]].push_back(v[i]);
      g[v[i]].push_back(u[i]);
   }
   HLD hld(g);
   hld.build();
   vi edgetoin(m);
   rep(i, m) {
      // 辺を子のinに対応させる
      int a = u[i], b = v[i];
      if(hld.par[b] == a) swap(a, b);
      edgetoin[i] = hld.in[a];
   }
   SegmentTree<S, op, e> seg(n);
   rep(i, m) { seg.set(edgetoin[i], w[i]); }
   auto query = [&](int l, int r) { return seg.prod(l, r); };
   int q;
   cin >> q;
   rep(qi, q) {
      int t;
      cin >> t;
      if(t == 1) {
         int i, x;
         cin >> i >> x;
         i--;
         w[i] = x;
         seg.set(edgetoin[i], w[i]);
      } else {
         int a, b;
         cin >> a >> b;
         a--;
         b--;
         auto res = hld.query(a, b, e(), query, op, true);
         cout << res << endl;
      }
   }
}
int main() {
   cin.tie(0);
   ios::sync_with_stdio(false);
   int t = 1;  // cin >>t;
   rep(test, t) solve(test);
}
```

lca.hpp

md5: 37018c

```
// https://github.com/YNUCPC/ynu-icpc-
library/blob/main/src/graph/lca.hpp
struct LCA {
   vector<vector<int>> parent;
   vector<int> depth;
   LCA() {}
   LCA(const vector<vector<int>>& G, int r = 0) { init(G, r); }

   void init(const vector<vector<int>>& G, int r = 0) {
      int V = (int)G.size();
      int h = 1;
      while((1 << h) < V) ++h;
      parent.assign(h, vector<int>(V, -1));
      depth.assign(V, -1);
      dfs(G, r, -1, 0);
      for(int i = 0; i + 1 < (int)parent.size(); ++i) {
         for(int v = 0; v < V; ++v) {
            if(parent[i][v] != -1) { parent[i + 1][v] =
parent[i][parent[i][v]]; }
         }
      }
   }

   void dfs(const vector<vector<int>>& G, int v, int p, int d)
{
      parent[0][v] = p;
      depth[v] = d;
      for(auto e : G[v])
         if(e != p) dfs(G, e, v, d + 1);
   }

   int query(int u, int v) {
      if(depth[u] > depth[v]) swap(u, v);
      for(int i = 0; i < (int)parent.size(); ++i) {
         if((depth[v] - depth[u]) & (1 << i)) v = parent[i][v];
      }
      if(u == v) return u;
      for(int i = (int)parent.size() - 1; i >= 0; --i) {
         if(parent[i][u] != parent[i][v]) {
            u = parent[i][u];
            v = parent[i][v];
         }
      }
      return parent[0][u];
   }

   int dist(int u, int v) { return depth[u] + depth[v] - 2 *
depth[query(u, v)]; }

   bool is_on_path(int u, int v, int x) { return dist(u, x) +
dist(x, v) == dist(u, v); }
};
```

lca_me.hpp

md5: 0e5db2

```
const int K = 22;
vve g;  // gだけ初期化
vvi parent;
vi depth;
vll Dist;
void lca_dfs(int v, int p = -1, int d = 0) {
   parent[v][0] = p;
   depth[v] = d;
   for(auto u : g[v]) {
      if(u.to == p) continue;
      Dist[u.to] = Dist[v] + u.cost;
      lca_dfs(u.to, v, d + 1);
   }
}
void lca_init(int n) {
   parent.resize(n, vi(K + 5));
   depth.resize(n);
   Dist.resize(n);
   lca_dfs(0, -1, 0);
   rep(k, K - 1) {
      rep(v, n) {
         if(parent[v][k] < 0) parent[v][k + 1] = -1;
         else parent[v][k + 1] = parent[parent[v][k]][k];
      }
   }
}
int lca(int u, int v) {
   if(depth[u] > depth[v]) swap(u, v);
   rep(k, K) {
      if(((depth[v] - depth[u]) >> k) & 1) { v = parent[v][k];
}
   }
   if(u == v) return v;
   for(int k = K - 1; k >= 0; k--) {
      if(parent[u][k] != parent[v][k]) {
         u = parent[u][k];
         v = parent[v][k];
      }
   }
   return parent[u][0];
}
int getAnce(int v, ll k) {
   int now = v;
   rep(j, K) {
      if(now == -1) continue;
      if(k >> j & 1) { now = parent[now][j]; }
   }
   return now;
}
ll distacn_uv(int u, int v) { return Dist[u] + Dist[v] - 2 *
Dist[lca(u, v)]; }
```

lowlink.hpp

md5: 07c344

```cpp
//
https://github.com/KentaroMatsushita/icpc_library/blob/main/src/graph/lowlink.hpp
#define pii pair<int, int>
#define fore(e, v) for(auto&& e : v)
#define eb emplace_back
#define si(a) (int)(size(a))
template<typename G> struct LL {
   int n;
   const G g;
   vi ord, low, arti;
   vector<pii> bridge;

   LL(G g) : n(si(g)), g(g), ord(si(g), -1), low(si(g), -1) {
      int k = 0;
      rep(i, n) {
         if(ord[i] == -1) k = dfs(i, k, -1);
      }
   }

   int dfs(int x, int k, int p) {
      low[x] = (ord[x] = k++);
      int cnt = 0;
      bool is_arti = false, second = false;
      fore(e, g[x]) {
         if(ord[e] == -1) {
            cnt++;
            k = dfs(e, k, x);
            chmin(low[x], low[e]);
            is_arti |= (p != -1) && (low[e] >= ord[x]);
            if(ord[x] < low[e]) bridge.eb(minmax(x, e));
         } else if(e != p or second) {
            chmin(low[x], ord[e]);
         } else {
            second = true;
         }
      }
      is_arti |= p == -1 && cnt > 1;
      if(is_arti) arti.eb(x);
      return k;
   }
};
```

scc.hpp

md5: 9d4785

```cpp
//
https://github.com/KentaroMatsushita/icpc_library/blob/main/src/template/1template.hpp
#define fore(e, v) for(auto&& e : v)
#define eb emplace_back
#define si(a) (int)(size(a))
template<typename G> struct SCC {
   G g;
   vector<vi> rg;
   vi comp, ord, used;
   int num;  // 連結成分の数

   SCC(G g) : g(g), rg(si(g)), comp(si(g), -1), ord(si(g)),
used(si(g)) {
      rep(i, si(g)) fore(e, g[i]) rg[e].eb(i);
      build();
   };
   int operator[](int k) { return comp[k]; }
   void dfs(int x) {
      if(used[x]) return;
      used[x] = true;
      fore(e, g[x]) if(!used[e]) dfs(e);
      ord.eb(x);
   }
   void rdfs(int x, int cnt) {
      if(comp[x] != -1) return;
      comp[x] = cnt;
      fore(e, rg[x]) if(comp[e] == -1) rdfs(e, cnt);
   }
   void build() {
      rep(i, g.size()) dfs(i);
      reverse(all(ord));
      num = 0;
      fore(i, ord) if(comp[i] == -1) { rdfs(i, num), num++; }
   }
};
```

scc_yokohama.hpp

md5: cb50ac

```cpp
#pragma once
#include <bits/stdc++.h>
using namespace std;

struct SCC {
   int _n;
   struct edge {
      int to;
   };
   vector<pair<int, edge>> edges;

   template<class E> struct csr {
      vector<int> start;
      vector<E> elist;
      csr(int n, const vector<pair<int, E>>& edges) : start(n +
1), elist(edges.size()) {
         for(auto e : edges) start[e.first + 1]++;
         for(int i = 1; i <= n; i++) start[i] += start[i - 1];
         auto counter = start;
         for(auto e : edges) elist[counter[e.first]++] =
e.second;
      }
   };

   SCC(int n) : _n(n) {}
   SCC() : _n(0) {}

   int num_vertices() { return _n; }

   void add_edge(int from, int to) { edges.push_back({from,
{to}}); }

   // return pair of (# of scc, scc id)
   pair<int, vector<int>> scc_ids() {
      auto g = csr<edge>(_n, edges);
      int now_ord = 0, group_num = 0;
      vector<int> visited, low(_n), ord(_n, -1), ids(_n);
      visited.reserve(_n);
      auto dfs = [&](auto self, int v) -> void {
         low[v] = ord[v] = now_ord++;
         visited.push_back(v);
         for(int i = g.start[v]; i < g.start[v + 1]; i++) {
            auto to = g.elist[i].to;
            if(ord[to] == -1) {
               self(self, to);
               low[v] = min(low[v], low[to]);
            } else {
               low[v] = min(low[v], ord[to]);
            }
         }
         if(low[v] == ord[v]) {
            while(true) {
               int u = visited.back();
               visited.pop_back();
               ord[u] = _n;
               ids[u] = group_num;
               if(u == v) break;
            }
            group_num++;
         }
      };
      for(int i = 0; i < _n; i++)
         if(ord[i] == -1) dfs(dfs, i);
      for(auto& x : ids) x = group_num - 1 - x;
      return {group_num, ids};
   }

   vector<vector<int>> scc() {
      auto ids = scc_ids();
      int group_num = ids.first;
      vector<int> counts(group_num);
      for(auto x : ids.second) counts[x]++;
      vector<vector<int>> groups(ids.first);
      for(int i = 0; i < group_num; i++)
groups[i].reserve(counts[i]);
      for(int i = 0; i < _n; i++)
groups[ids.second[i]].push_back(i);
      return groups;
   }
};
```

flow

bipartite-matching.hpp

md5: 845322

```
//
https://github.com/KentaroMatsushita/icpc_library/blob/main/src/flow/bipartite-
matching.hpp
#define eb emplace_back
#define si(a) (int)(size(a))
#define fore(e, v) for(auto&& e : v)
struct Bimatch {
   vector<vi> g;
   vi d, mc, used, vv;
   Bimatch(int n, int m) : g(n), mc(m, -1), used(n) {}
   void add(int u, int v) { g[u].eb(v); }
   void bfs() {
      d.assign(si(g), -1);
      queue<int> q;
      rep(i, si(g)) {
         if(!used[i]) {
            q.emplace(i);
            d[i] = 0;
         }
      }
      while(!q.empty()) {
         int x = q.front();
         q.pop();
         fore(e, g[x]) {
            int c = mc[e];
            if(c >= 0 and d[c] == -1) {
               d[c] = d[x] + 1;
               q.emplace(c);
            }
         }
      }
   }

   bool dfs(int x) {
      vv[x] = true;
      fore(e, g[x]) {
         int c = mc[e];
         if(c < 0 or (!vv[c] and d[c] == d[x] + 1 and dfs(c)))
{
            mc[e] = x;
            used[x] = true;
            return true;
         }
      }
      return false;
   }
   int match() {
      int ret = 0;
      while(true) {
         bfs();
         vv.assign(si(g), false);
         int f = 0;
         rep(i, si(g)) if(!used[i] and dfs(i)) f++;
         if(!f) return ret;
         ret += f;
      }
   }
};
```

maxflow.hpp

md5: 7ddb68

```
// https://github.com/YNUCPC/ynu-icpc-
library/blob/main/src/flow/dinic.hpp
//  Dinic法 O(V^2E)
struct mf_graph {
   int V;                                        // 頂点数
   vector<vector<vector<long long>>> graph;  // グラフ
   vector<int> dis;                              // 始点からの距離
   vector<int> next;                             // 次に処理する頂点の
メモ
   mf_graph(int v) : V(v) { graph.resize(V); }
   void add_edge(int from, int to, long long capacity) {
      graph[from].push_back({to, capacity,
(int)graph[to].size()});
      graph[to].push_back({from, 0, (int)graph[from].size() -
1});
   }
   void bfs(int s) {
      dis.assign(V, -1);
      dis[s] = 0;
      deque<int> pos = {s};
      while(pos.size()) {
         int now = pos[0];
         pos.pop_front();
         for(auto& to : graph[now]) {
            if(dis[to[0]] < 0 and to[1] > 0) {
               dis[to[0]] = dis[now] + 1;
               pos.emplace_back(to[0]);
            }
         }
      }
   }
   long long dfs(int v, int t, long long f) {
      if(v == t) return f;
      for(int& i = next[v]; i < graph[v].size(); i++) {
         int to = graph[v][i][0];
         long long& cap = graph[v][i][1];
         int rev = graph[v][i][2];
         if(cap > 0 and dis[v] < dis[to]) {
            long long d = dfs(to, t, min(f, cap));
            if(d > 0) {
               cap -= d;
               graph[to][rev][1] += d;
               return d;
            }
         }
      }
      return 0;
   }
   long long max_flow(int s, int t) {
      long long flow = 0;
      while(1) {
         bfs(s);
         if(dis[t] < 0) return flow;
         next.assign(V, 0);
         long long f;
         while((f = dfs(s, t, LLONG_MAX)) > 0) flow += f;
      }
   }
};
```

mincostflow.hpp

md5: 70df9f

```
// https://github.com/YNUCPC/ynu-icpc-
library/blob/main/src/flow/mincostflow.hpp
//  最小費用流 O(FElogV)
struct MinCostFlow {
   int V;
   vector<vector<vector<long long>>> g;  // g[from] = {{to, 容
量, コスト, 逆辺のindex} ... }
   vector<long long> h, dis;             // ポテンシャル, 最短距離
   vector<int> prevv, preve;             // 直前の頂点, 辺

   MinCostFlow(int v) : V(v), g(v), dis(v), prevv(v), preve(v)
{}

   void add_edge(int u, int v, long long c, long long d) {
      g[u].push_back({v, c, d, (int)g[v].size()});
      g[v].push_back({u, 0, -d, (int)g[u].size() - 1});
   }

   long long min_cost_flow(int s, int t, long long f) {
      long long res = 0;
      h.assign(V, 0);
      using Q = pair<long long, int>;
      while(f > 0) {
         priority_queue<Q, vector<Q>, greater<Q>> que;
         dis.assign(V, LLONG_MAX);
         dis[s] = 0;
         que.push({0, s});
         while(que.size()) {
            Q q = que.top();
            int v = q.second;
            que.pop();
            if(dis[v] < q.first) continue;
            for(int i = 0; i < g[v].size(); i++) {
               auto edge = g[v][i];
               int to = edge[0];
               long long cap = edge[1], cost = edge[2];
               if(cap > 0 and dis[to] > dis[v] + cost + h[v] -
h[to]) {
                  dis[to] = dis[v] + cost + h[v] - h[to];
                  prevv[to] = v;
                  preve[to] = i;
                  que.push({dis[to], to});
```

```
            }
         }
      }
      if(dis[t] == LLONG_MAX) return -1;
      for(int i = 0; i < V; i++) h[i] += dis[i];
      long long d = f;
      for(int i = t; i != s; i = prevv[i]) d = min(d,
g[prevv[i]][preve[i]][1]);
      f -= d;
      res += d * h[t];
      for(int i = t; i != s; i = prevv[i]) {
         auto& edge = g[prevv[i]][preve[i]];
         edge[1] -= d;
         g[i][edge[3]][1] += d;
      }
   }
   return res;
  }
};
```

燃やす埋める.md

変形前の制約	変形後の制約
x が 0 のとき z 失う	(x, T, z)
x が 0 のとき z 得る	無条件で z 得る; (S, x, z)
x が 1 のとき z 失う	(S, x, z)
x が 1 のとき z 得る	無条件で z 得る; (x, T, z)
$x, y, \ldots$ がすべて 0 のとき z 得る	無条件で z 得る; $(S, w, z), (w, x, \infty), (w, y, \infty)$
$x, y, \ldots$ がすべて 1 のとき z 得る	無条件で z 得る; $(w, T, z), (x, w, \infty), (y, w, \infty)$

string

KMP.hpp

md5: 886c63

```
// kmp[i] := max{ l ≤ i | s[:l] == s[(i+1)-l:i+1] }
// abacaba -> 0010123
auto KMP(string s) {
   vector<ll> p(sz(s));
   rep(i, 1, sz(s)) {
      ll g = p[i - 1];
      while(g && s[i] != s[g]) g = p[g - 1];
      p[i] = g + (s[i] == s[g]);
   }
   return p;
}
```

Manacher.hpp

md5: 5882fb

```
// 各位置での回文半径を求める
// aaabaaa -> 1214121
// 偶数長の回文を含めて直径を知るには, N+1 個の $ を挿入して 1 を引く
// $a$a$a$b$a$a$a$ -> 123432181234321
auto manacher(string s) {
   ll n = sz(s), i = 0, j = 0;
   vector<ll> r(n);
   while(i < n) {
      while(i >= j && i + j < n && s[i - j] == s[i + j]) j++;
      r[i] = j;
      ll k = 1;
      while(i >= k && i + k < n && k + r[i - k] < j) {
         r[i + k] = r[i - k];
         k++;
      }
      i += k, j -= k;
   }
   return r;
}
```

RollingHash.hpp

md5: adb8d3

```
// using u64 = uint64_t;
const u64 mod = INF;
u64 add(u64 a, u64 b) {
   a += b;
   if(a >= mod) a -= mod;
   return a;
}
u64 mul(u64 a, u64 b) {
   auto c = (__uint128_t)a * b;
   return add(c >> 61, c & mod);
}
random_device rnd;
const u64 r = ((u64)rnd() << 32 | rnd()) % mod;
struct RH {
   ll n;
   vector<u64> hs, pw;
   RH(string s) : n(sz(s)), hs(n + 1), pw(n + 1, 1) {
      rep(i, 0, n) {
         pw[i + 1] = mul(pw[i], r);
         hs[i + 1] = add(mul(hs[i], r), s[i]);
      }
   }
   u64 get(ll l, ll r) const { return add(hs[r], mod -
mul(hs[l], pw[r - l])); }
};
```

SuffixArray_spped_star.hpp

md5: 2a0504

```
// lcp先頭変なのはいっていそう
//  returns pair{sa, lcp}
//  sa 長さ n : s[sa[0]:] < s[sa[1]:] < … < s[sa[n-1]:]
//  lcp 長さ n-1 : lcp[i] = LCP(s[sa[i]:], s[sa[i+1]:])
#define si(a) (int)(size(a))
auto SA(string s) {
   ll n = si(s) + 1, lim = 256;
   // assert(lim > ranges::max(s));
   vector<ll> sa(n), lcp(n), x(all(s) + 1), y(n), ws(max(n,
lim)), rk(n);
   iota(all(sa), 0);
   for(ll j = 0, p = 0; p < n; j = max(1LL, j * 2), lim = p) {
      p = j;
      iota(all(y), n - j);
      repi(i, 0, n) if(sa[i] >= j) y[p++] = sa[i] - j;
      fill(all(ws), 0);
      repi(i, 0, n) ws[x[i]]++;
      repi(i, 1, lim) ws[i] += ws[i - 1];
      for(ll i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y);
      p = 1;
      x[sa[0]] = 0;
      repi(i, 1, n) {
         ll a = sa[i - 1], b = sa[i];
         x[b] = (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1
: p++;
      }
   }
   repi(i, 1, n) rk[sa[i]] = i;
   for(ll i = 0, k = 0; i < n - 1; lcp[rk[i++]] = k) {
      if(k) k--;
      while(s[i + k] == s[sa[rk[i] - 1] + k]) k++;
   }
   sa.erase(begin(sa));
   lcp.erase(begin(lcp));
   return pair{sa, lcp};
}
void solve(int test) {
   string s;
   cin >> s;
   int n = s.size();
   auto re = SA(s);
   auto sa = re.first, lcp = re.second;
   reverse(all(lcp));
   lcp.pop_back();
   reverse(all(lcp));
}
```

Zalgorithm.hpp

md5: b20b04

```
// Z[i] := LCP(s, s[i:])
// abacaba -> 7010301
auto Z(string s) {
   ll n = sz(s), l = -1, r = -1;
   vector<ll> z(n, n);
   rep(i, 1, n) {
      ll& x = z[i] = i < r ? min(r - i, z[i - l]) : 0;
      while(i + x < n && s[i + x] == s[x]) x++;
      if(i + x > r) l = i, r = i + x;
   }
   return z;
}
```

kaibun.hpp

md5: c3ad5a

```
vi manachar(string S) {
   int i = 0, j = 0;
   vi R(S.size());
   while(i < S.size()) {
      while(i - j >= 0 && i + j < S.size() && S[i - j] == S[i + j]) ++j;
      R[i] = j;
      int k = 1;
      while(i - k >= 0 && k + R[i - k] < j) R[i + k] = R[i - k], ++k;
      i += k;
      j -= k;
   }
   return R;
}
// 奇数,偶数の最大半径
pair<vi, vi> kaibun(string s) {
   vi l = manachar(s);
   string t = "";
   int n = s.size();
   rep(i, n) {
      t += s[i];
      if(i != n - 1) t += '@';
   }
   vi v = manachar(t);
   vi r;
   for(int i = 1; i < v.size(); i += 2) {
      if(i >= v.size()) break;
      r.push_back(v[i] / 2);
   }
   return make_pair(l, r);
}
```

lcp_array.hpp

md5: 2c8f78

```
// https://github.com/YNUCPC/ynu-icpc-library/blob/main/src/string/lcp_array.hpp
//  lcp[i] = sa[i]とsa[i+1]の common prefix の長さ
vector<int> lcp_array(const string& s, const vector<int>& sa) {
   int n = sa.size();
   vector<int> lcp(n), rank(n);  // rank[i] = s[i,n)の辞書順位
   for(int i = 0; i < n; i++) rank[sa[i]] = i;
   for(int i = 0, len = 0; i < n; i++) {
      if(len > 0) len--;
      if(rank[i] == n - 1) continue;
      int j = sa[rank[i] + 1];  // s[i,n)より辞書順で1つ大きいsuffixの先頭位置
      while(max(i, j) + len < int(s.size()) && s[i + len] == s[j + len]) len++;
      lcp[rank[i]] = len;
   }
   return lcp;
}

/*
string s; cin >> s;
int n=s.size();
auto sa=suffix_array(s);
auto lcp=lcp_array(s,sa);
vi id(n+1);
rep(i,sa.size())id[sa[i]]=i;
SegmentTree<S,op,e> seg(lcp);
vvm dp(n,vm(n));
repi(l,1,n)repi(r,l+1,n){
int len=seg.prod(min(id[l],id[r]),max(id[l],id[r]));
*/
```

rolling_hash.hpp

md5: a0ca36

```
// https://github.com/YNUCPC/ynu-icpc-library/blob/main/src/string/rolling_hash.hpp
struct RollingHash {
   static constexpr int M = 2;
   static constexpr long long MODS[M] = {999999937, 1000000007};
   static constexpr long long BASE = 9973;
   vector<long long> powb[M], hash[M];
   int n;
   RollingHash() {}
   RollingHash(const string& str) { init(str); }
   void init(const string& str) {
      n = str.size();
      for(int k = 0; k < M; k++) {
         powb[k].resize(n + 1, 1);
         hash[k].resize(n + 1, 0);
         for(int i = 0; i < n; i++) {
            hash[k][i + 1] = (hash[k][i] * BASE + str[i]) % MODS[k];
            powb[k][i + 1] = powb[k][i] * BASE % MODS[k];
         }
      }
   }
   // get hash str[l,r)
   long long get(int l, int r, int k) {
      long long res = hash[k][r] - hash[k][l] * powb[k][r - l] % MODS[k];
      if(res < 0) res += MODS[k];
      return res;
   }
};

bool match(RollingHash& rh1, int l1, int r1, RollingHash& rh2, int l2, int r2) {
   bool res = true;
   for(int k = 0; k < RollingHash::M; k++) { res &= rh1.get(l1, r1, k) == rh2.get(l2, r2, k); }
   return res;
}
```

suffix_array.hpp

md5: 1ef41f

```
// https://github.com/YNUCPC/ynu-icpc-library/blob/main/src/string/suffix_array.hpp
//  長さn+1に注意,最初がn?
vector<int> suffix_array(const string& str) {
   int n = str.size();
   vector<int> sa(n + 1), rank(n + 1, -1);  // sa[i] = 辞書順でi番目であるsuffixの開始位置
   iota(sa.begin(), sa.end(), 0);
   for(int i = 0; i < n; i++) rank[i] = str[i];
   int k;
   auto comp = [&](const int& i, const int& j) {
      if(rank[i] != rank[j]) {
         return rank[i] < rank[j];
      } else {
         int ri = i + k <= n ? rank[i + k] : -1;
         int rj = j + k <= n ? rank[j + k] : -1;
         return ri < rj;
      }
   };
   for(k = 1; k <= n; k <<= 1) {
      sort(sa.begin(), sa.end(), comp);
      vector<int> tmp(n + 1, 0);
      for(int i = 0; i < n; i++) {
         tmp[sa[i + 1]] = tmp[sa[i]];
         if(comp(sa[i], sa[i + 1])) tmp[sa[i + 1]]++;
      }
      rank = tmp;
   }
   return sa;
}

// 文字列sに文字列tに含まれているか判定する
bool contain(const string& s, const string& t, vector<int>& sa)
{
   int l = 0, r = int(s.size());
   while(r - l > 1) {
      int mid = (l + r) / 2;
      if(s.substr(sa[mid], t.size()) < t) {
         l = mid;
      } else {
         r = mid;
      }
   }
   return s.substr(sa[r], t.size()) == t;
}
```

z_algorithm.hpp

md5: 146991

```
// https://github.com/YNUCPC/ynu-icpc-library/blob/main/src/string/z_algorithm.hpp
vector<int> z_algorithm(string& s) {
   int n = int(s.size());
   vector<int> z(n);
   z[0] = n;
   for(int i = 1, l = 0, r = 0; i < n; i++) {
      int& k = z[i];
```

```
      k = (r <= i ? 0 : min(r - i, z[i - l]));
      while(i + k < n && s[k] == s[i + k]) k++;
      if(r < i + k) l = i, r = i + k;
   }
   return z;
}
```

geometry

memo

Bi-Connected-Components.hpp

md5: eddd4d

```
template<typename G> struct BiConnectedComponents : LowLink<G>
{
   using LL = LowLink<G>;

   vector<int> used;
   vector<vector<pair<int, int> > > bc;
   vector<pair<int, int> > tmp;

   BiConnectedComponents(const G& g) : LL(g) {}

   void dfs(int idx, int par) {
      used[idx] = true;
      for(auto& to : this->g[idx]) {
         if(to == par) continue;
         if(!used[to] || this->ord[to] < this->ord[idx]) {
tmp.emplace_back(minmax(idx, to)); }
         if(!used[to]) {
            dfs(to, idx);
            if(this->low[to] >= this->ord[idx]) {
               bc.emplace_back();
               for(;;) {
                  auto e = tmp.back();
                  bc.back().emplace_back(e);
                  tmp.pop_back();
                  if(e.first == min(idx, to) && e.second ==
max(idx, to)) { break; }
               }
            }
         }
      }
   }

   void build() override {
      LL::build();
      used.assign(this->g.size(), 0);
      for(int i = 0; i < used.size(); i++) {
         if(!used[i]) dfs(i, -1);
      }
   }
};
```

使用例

```
template<int mod> struct ModInt {
   int x;

   ModInt() : x(0) {}

   ModInt(int64_t y) : x(y >= 0 ? y % mod : (mod - (-y) % mod)
% mod) {}

   ModInt& operator+=(const ModInt& p) {
      if((x += p.x) >= mod) x -= mod;
      return *this;
   }

   ModInt& operator-=(const ModInt& p) {
      if((x += mod - p.x) >= mod) x -= mod;
      return *this;
   }

   ModInt& operator*=(const ModInt& p) {
      x = (int)(1LL * x * p.x % mod);
      return *this;
   }

   ModInt& operator/=(const ModInt& p) {
      *this *= p.inverse();
      return *this;
   }

   ModInt operator-() const { return ModInt(-x); }

   ModInt operator+(const ModInt& p) const { return
ModInt(*this) += p; }

   ModInt operator-(const ModInt& p) const { return
ModInt(*this) -= p; }

   ModInt operator*(const ModInt& p) const { return
ModInt(*this) *= p; }

   ModInt operator/(const ModInt& p) const { return
ModInt(*this) /= p; }

   bool operator==(const ModInt& p) const { return x == p.x; }

   bool operator!=(const ModInt& p) const { return x != p.x; }

   ModInt inverse() const {
      int a = x, b = mod, u = 1, v = 0, t;
      while(b > 0) {
         t = a / b;
         swap(a -= t * b, b);
         swap(u -= t * v, v);
      }
      return ModInt(u);
   }

   ModInt pow(int64_t n) const {
      ModInt ret(1), mul(x);
      while(n > 0) {
         if(n & 1) ret *= mul;
         mul *= mul;
         n >>= 1;
      }
      return ret;
   }

   friend ostream& operator<<(ostream& os, const ModInt& p) {
return os << p.x; }

   friend istream& operator>>(istream& is, ModInt& a) {
      int64_t t;
      is >> t;
      a = ModInt<mod>(t);
      return (is);
   }
};

using modint = ModInt<mod>;

template<typename T> struct Combination {
   vector<T> _fact, _rfact;

   Combination(int sz) : _fact(sz + 1), _rfact(sz + 1) {
      _fact[0] = _rfact[sz] = 1;
      for(int i = 1; i <= sz; i++) _fact[i] = _fact[i - 1] * i;
      _rfact[sz] /= _fact[sz];
      for(int i = sz - 1; i >= 0; i--) _rfact[i] = _rfact[i +
1] * (i + 1);
   }

   inline T fact(int k) const { return _fact[k]; }

   inline T rfact(int k) const { return _rfact[k]; }

   T P(int n, int r) const {
      if(r < 0 || n < r) return 0;
      return fact(n) * rfact(n - r);
   }

   T C(int p, int q) const {
      if(q < 0 || p < q) return 0;
      return fact(p) * rfact(q) * rfact(p - q);
   }

   T H(int n, int r) const {
      if(n < 0 || r < 0) return (0);
      return r == 0 ? 1 : C(n + r - 1, r);
   }
};

int64_t euler_phi(int64_t n) {
   int64_t ret = n;
   for(int64_t i = 2; i * i <= n; i++) {
      if(n % i == 0) {
         ret -= ret / i;
         while(n % i == 0) n /= i;
```

```
      }
   }
   if(n > 1) ret -= ret / n;
   return ret;
}

int main() {
   int N, M, K;
   cin >> N >> M >> K;
   UnWeightedGraph g(N);
   Combination<modint> beet(101010);
   while(M--) {
      int x, y;
      cin >> x >> y;
      --x, --y;
      g[x].push_back(y);
      g[y].push_back(x);
   }
   BiConnectedComponents<UnWeightedGraph> bcc(g);
   bcc.build();
   modint ret = 1;
   for(auto& vs : bcc.bc) {
      set<int> vertex;
      for(auto& p : vs) vertex.emplace(p.first);
      for(auto& p : vs) vertex.emplace(p.second);
      if(vertex.size() == vs.size() + 1) {
         ret *= K;
      } else if(vertex.size() == vs.size()) {
         modint add = 0;
         for(int i = 1; i <= vertex.size(); i++) {
            if(vertex.size() % i == 0) { add += modint(K).pow(vertex.size() / i) * euler_phi(i); }
         }
         add /= vertex.size();
         ret *= add;
      } else {
         ret *= beet.H(K, vs.size());
      }
   }
   cout << ret << endl;
}
```

Dynamic-Li-Chao-Tree.hpp

md5: 541e42

```
/**
 * @brief Dynamic-Li-Chao-Tree
 *
 */
template<typename T, T x_low, T x_high, T id> struct DynamicLiChaoTree {
   struct Line {
      T a, b;

      Line(T a, T b) : a(a), b(b) {}

      inline T get(T x) const { return a * x + b; }
   };

   struct Node {
      Line x;
      Node *l, *r;

      Node(const Line& x) : x{x}, l{nullptr}, r{nullptr} {}
   };

   Node* root;

   DynamicLiChaoTree() : root{nullptr} {}

   Node* add_line(Node* t, Line& x, const T& l, const T& r, const T& x_l, const T& x_r) {
      if(!t) return new Node(x);

      T t_l = t->x.get(l), t_r = t->x.get(r);

      if(t_l <= x_l && t_r <= x_r) {
         return t;
      } else if(t_l >= x_l && t_r >= x_r) {
         t->x = x;
         return t;
      } else {
         T m = (l + r) / 2;
         if(m == r) --m;
         T t_m = t->x.get(m), x_m = x.get(m);
         if(t_m > x_m) {
            swap(t->x, x);
            if(x_l >= t_l) t->l = add_line(t->l, x, l, m, t_l, t_m);
            else t->r = add_line(t->r, x, m + 1, r, t_m + x.a, t_r);
         } else {
            if(t_l >= x_l) t->l = add_line(t->l, x, l, m, x_l, x_m);
            else t->r = add_line(t->r, x, m + 1, r, x_m + x.a, x_r);
         }
         return t;
      }
   }

   void add_line(const T& a, const T& b) {
      Line x(a, b);
      root = add_line(root, x, x_low, x_high, x.get(x_low), x.get(x_high));
   }

   Node* add_segment(Node* t, Line& x, const T& a, const T& b, const T& l, const T& r, const T& x_l, const T& x_r) {
      if(r < a || b < l) return t;
      if(a <= l && r <= b) {
         Line y{x};
         return add_line(t, y, l, r, x_l, x_r);
      }
      if(t) {
         T t_l = t->x.get(l), t_r = t->x.get(r);
         if(t_l <= x_l && t_r <= x_r) return t;
      } else {
         t = new Node(Line(0, id));
      }
      T m = (l + r) / 2;
      if(m == r) --m;
      T x_m = x.get(m);
      t->l = add_segment(t->l, x, a, b, l, m, x_l, x_m);
      t->r = add_segment(t->r, x, a, b, m + 1, r, x_m + x.a, x_r);
      return t;
   }

   void add_segment(const T& l, const T& r, const T& a, const T& b) {
      Line x(a, b);
      root = add_segment(root, x, l, r - 1, x_low, x_high, x.get(x_low), x.get(x_high));
   }

   T query(const Node* t, const T& l, const T& r, const T& x) const {
      if(!t) return id;
      if(l == r) return t->x.get(x);
      T m = (l + r) / 2;
      if(m == r) --m;
      if(x <= m) return min(t->x.get(x), query(t->l, l, m, x));
      else return min(t->x.get(x), query(t->r, m + 1, r, x));
   }

   T query(const T& x) const { return query(root, x_low, x_high, x); }
};
/*
x_low には query() で与える
x の最小値,
x_high には x の最大値,
id は単位元(十分大きい値) を指定する. すべての直線について x_low, x_high がオーバーフローしないとき意図した動作をする.
最大値クエリは最小値クエリに帰着できて, add_line(-a, -b), -query(x) とすればよい.
add_line(a, b): 直線 ax+bを追加する.
add_segment(l, r, a, b): 区間 [l,r) に ax+bを追加する.
query(x): ax+b の最小値を求める.
```

Primes.md

素数の個数

n	10^2	10^3	10^4	10^5	10^6	10^7	10^8	10^9	10^{10}
$\pi(n)$	25	168	1229	9592	78498	664579	5.76e+6	5.08e+7	4.55e+8

高度合成数

$\leq n$	10^3	10^4	10^5	10^6	10^7	10^8	10^9
x	840	7560	83160	720720	8648640	73513440	735134400
$d^0(x)$	32	64	128	240	448	768	1344

$\leq n$	10^{10}	10^{11}	10^{12}	10^{13}	10^{14}	10^{15}	10^{16}	10^{17}	10^{18}
$d^0(x)$	2304	4032	6720	10752	17280	26880	41472	64512	103680

素数階乗

n	2	3	5	7	11	13	17	19	23	29
$n\#$	2	6	30	210	2310	30030	510510	9.70e+6	2.23e+8	6.47e+9

階乗

4!	5!	6!	7!	8!	9!	10!	11!	12!	13!
24	120	720	5040	40320	362880	3.63e+6	3.99e+7	4.79e+8	6.23e+9

bostanmori.hpp

md5: 08194c

```cpp
// modint
template<int MOD> struct Fp {
   long long val;
   constexpr Fp(long long v = 0) noexcept : val(v % MOD) {
      if(val < 0) val += MOD;
   }
   constexpr int getmod() const { return MOD; }
   constexpr Fp operator-() const noexcept { return val ? MOD -
val : 0; }
   constexpr Fp operator+(const Fp& r) const noexcept { return
Fp(*this) += r; }
   constexpr Fp operator-(const Fp& r) const noexcept { return
Fp(*this) -= r; }
   constexpr Fp operator*(const Fp& r) const noexcept { return
Fp(*this) *= r; }
   constexpr Fp operator/(const Fp& r) const noexcept { return
Fp(*this) /= r; }
   constexpr Fp& operator+=(const Fp& r) noexcept {
      val += r.val;
      if(val >= MOD) val -= MOD;
      return *this;
   }
   constexpr Fp& operator-=(const Fp& r) noexcept {
      val -= r.val;
      if(val < 0) val += MOD;
      return *this;
   }
   constexpr Fp& operator*=(const Fp& r) noexcept {
      val = val * r.val % MOD;
      return *this;
   }
   constexpr Fp& operator/=(const Fp& r) noexcept {
      long long a = r.val, b = MOD, u = 1, v = 0;
      while(b) {
         long long t = a / b;
         a -= t * b, swap(a, b);
         u -= t * v, swap(u, v);
      }
      val = val * u % MOD;
      if(val < 0) val += MOD;
      return *this;
   }
   constexpr bool operator==(const Fp& r) const noexcept {
return this->val == r.val; }
   constexpr bool operator!=(const Fp& r) const noexcept {
return this->val != r.val; }
   friend constexpr istream& operator>>(istream& is, Fp<MOD>&
x) noexcept {
      is >> x.val;
      x.val %= MOD;
      if(x.val < 0) x.val += MOD;
      return is;
   }
   friend constexpr ostream& operator<<(ostream& os, const
Fp<MOD>& x) noexcept { return os << x.val; }
   friend constexpr Fp<MOD> modpow(const Fp<MOD>& r, long long
n) noexcept {
      if(n == 0) return 1;
      if(n < 0) return modpow(modinv(r), -n);
      auto t = modpow(r, n / 2);
      t = t * t;
      if(n & 1) t = t * r;
      return t;
   }
   friend constexpr Fp<MOD> modinv(const Fp<MOD>& r) noexcept {
      long long a = r.val, b = MOD, u = 1, v = 0;
      while(b) {
         long long t = a / b;
         a -= t * b, swap(a, b);
         u -= t * v, swap(u, v);
      }
      return Fp<MOD>(u);
   }
};

namespace NTT {
long long modpow(long long a, long long n, int mod) {
   long long res = 1;
   while(n > 0) {
      if(n & 1) res = res * a % mod;
      a = a * a % mod;
      n >>= 1;
   }
   return res;
}

long long modinv(long long a, int mod) {
   long long b = mod, u = 1, v = 0;
   while(b) {
      long long t = a / b;
      a -= t * b, swap(a, b);
      u -= t * v, swap(u, v);
   }
   u %= mod;
   if(u < 0) u += mod;
   return u;
}

int calc_primitive_root(int mod) {
   if(mod == 2) return 1;
   if(mod == 167772161) return 3;
   if(mod == 469762049) return 3;
   if(mod == 754974721) return 11;
   if(mod == 998244353) return 3;
   int divs[20] = {};
   divs[0] = 2;
   int cnt = 1;
   long long x = (mod - 1) / 2;
   while(x % 2 == 0) x /= 2;
   for(long long i = 3; i * i <= x; i += 2) {
      if(x % i == 0) {
         divs[cnt++] = i;
         while(x % i == 0) x /= i;
      }
   }
   if(x > 1) divs[cnt++] = x;
   for(int g = 2;; g++) {
      bool ok = true;
      for(int i = 0; i < cnt; i++) {
         if(modpow(g, (mod - 1) / divs[i], mod) == 1) {
            ok = false;
            break;
         }
      }
      if(ok) return g;
   }
}

int get_fft_size(int N, int M) {
   int size_a = 1, size_b = 1;
   while(size_a < N) size_a <<= 1;
   while(size_b < M) size_b <<= 1;
   return max(size_a, size_b) << 1;
}

// number-theoretic transform
template<class mint> void trans(vector<mint>& v, bool inv =
false) {
   if(v.empty()) return;
   int N = (int)v.size();
   int MOD = v[0].getmod();
   int PR = calc_primitive_root(MOD);
   static bool first = true;
   static vector<long long> vbw(30), vibw(30);
   if(first) {
      first = false;
      for(int k = 0; k < 30; ++k) {
         vbw[k] = modpow(PR, (MOD - 1) >> (k + 1), MOD);
```

```
            vibw[k] = modinv(vbw[k], MOD);
        }
    }
    for(int i = 0, j = 1; j < N - 1; j++) {
        for(int k = N >> 1; k > (i ^= k); k >>= 1);
        if(i > j) swap(v[i], v[j]);
    }
    for(int k = 0, t = 2; t <= N; ++k, t <<= 1) {
        long long bw = vbw[k];
        if(inv) bw = vibw[k];
        for(int i = 0; i < N; i += t) {
            mint w = 1;
            for(int j = 0; j < t / 2; ++j) {
                int j1 = i + j, j2 = i + j + t / 2;
                mint c1 = v[j1], c2 = v[j2] * w;
                v[j1] = c1 + c2;
                v[j2] = c1 - c2;
                w *= bw;
            }
        }
    }
    if(inv) {
        long long invN = modinv(N, MOD);
        for(int i = 0; i < N; ++i) v[i] = v[i] * invN;
    }
}

// for garner
static constexpr int MOD0 = 754974721;
static constexpr int MOD1 = 167772161;
static constexpr int MOD2 = 469762049;
using mint0 = Fp<MOD0>;
using mint1 = Fp<MOD1>;
using mint2 = Fp<MOD2>;
static const mint1 imod0 = 95869806;    // modinv(MOD0, MOD1);
static const mint2 imod1 = 104391568;   // modinv(MOD1, MOD2);
static const mint2 imod01 = 187290749;  // imod1 / MOD0;

// small case (T = mint, long long)
template<class T> vector<T> naive_mul(const vector<T>& A, const
vector<T>& B) {
    if(A.empty() || B.empty()) return {};
    int N = (int)A.size(), M = (int)B.size();
    vector<T> res(N + M - 1);
    for(int i = 0; i < N; ++i)
        for(int j = 0; j < M; ++j) res[i + j] += A[i] * B[j];
    return res;
}

// mint
template<class mint> vector<mint> mul(const vector<mint>& A,
const vector<mint>& B) {
    if(A.empty() || B.empty()) return {};
    int N = (int)A.size(), M = (int)B.size();
    if(min(N, M) < 30) return naive_mul(A, B);
    int MOD = A[0].getmod();
    int size_fft = get_fft_size(N, M);
    if(MOD == 998244353) {
        vector<mint> a(size_fft), b(size_fft), c(size_fft);
        for(int i = 0; i < N; ++i) a[i] = A[i];
        for(int i = 0; i < M; ++i) b[i] = B[i];
        trans(a), trans(b);
        vector<mint> res(size_fft);
        for(int i = 0; i < size_fft; ++i) res[i] = a[i] * b[i];
        trans(res, true);
        res.resize(N + M - 1);
        return res;
    }
    vector<mint0> a0(size_fft, 0), b0(size_fft, 0), c0(size_fft,
0);
    vector<mint1> a1(size_fft, 0), b1(size_fft, 0), c1(size_fft,
0);
    vector<mint2> a2(size_fft, 0), b2(size_fft, 0), c2(size_fft,
0);
    for(int i = 0; i < N; ++i) a0[i] = A[i].val, a1[i] =
A[i].val, a2[i] = A[i].val;
    for(int i = 0; i < M; ++i) b0[i] = B[i].val, b1[i] =
B[i].val, b2[i] = B[i].val;
    trans(a0), trans(a1), trans(a2), trans(b0), trans(b1),
trans(b2);
    for(int i = 0; i < size_fft; ++i) {
        c0[i] = a0[i] * b0[i];
        c1[i] = a1[i] * b1[i];
        c2[i] = a2[i] * b2[i];
    }
    trans(c0, true), trans(c1, true), trans(c2, true);
    static const mint mod0 = MOD0, mod01 = mod0 * MOD1;
    vector<mint> res(N + M - 1);
    for(int i = 0; i < N + M - 1; ++i) {
        int y0 = c0[i].val;
        int y1 = (imod0 * (c1[i] - y0)).val;
        int y2 = (imod01 * (c2[i] - y0) - imod1 * y1).val;
        res[i] = mod01 * y2 + mod0 * y1 + y0;
    }
    return res;
}

// long long
vector<long long> mul_ll(const vector<long long>& A, const
vector<long long>& B) {
    if(A.empty() || B.empty()) return {};
    int N = (int)A.size(), M = (int)B.size();
    if(min(N, M) < 30) return naive_mul(A, B);
    int size_fft = get_fft_size(N, M);
    vector<mint0> a0(size_fft, 0), b0(size_fft, 0), c0(size_fft,
0);
    vector<mint1> a1(size_fft, 0), b1(size_fft, 0), c1(size_fft,
0);
    vector<mint2> a2(size_fft, 0), b2(size_fft, 0), c2(size_fft,
0);
    for(int i = 0; i < N; ++i) a0[i] = A[i], a1[i] = A[i], a2[i]
= A[i];
    for(int i = 0; i < M; ++i) b0[i] = B[i], b1[i] = B[i], b2[i]
= B[i];
    trans(a0), trans(a1), trans(a2), trans(b0), trans(b1),
trans(b2);
    for(int i = 0; i < size_fft; ++i) {
        c0[i] = a0[i] * b0[i];
        c1[i] = a1[i] * b1[i];
        c2[i] = a2[i] * b2[i];
    }
    trans(c0, true), trans(c1, true), trans(c2, true);
    static const long long mod0 = MOD0, mod01 = mod0 * MOD1;
    vector<long long> res(N + M - 1);
    for(int i = 0; i < N + M - 1; ++i) {
        int y0 = c0[i].val;
        int y1 = (imod0 * (c1[i] - y0)).val;
        int y2 = (imod01 * (c2[i] - y0) - imod1 * y1).val;
        res[i] = mod01 * y2 + mod0 * y1 + y0;
    }
    return res;
}
};  // namespace NTT

// Formal Power Series
template<typename mint> struct FPS : vector<mint> {
    using vector<mint>::vector;

    // constructor
    FPS(const vector<mint>& r) : vector<mint>(r) {}

    // core operator
    inline FPS pre(int siz) const { return FPS(begin(*this),
begin(*this) + min((int)this->size(), siz)); }
    inline FPS rev() const {
        FPS res = *this;
        reverse(begin(res), end(res));
        return res;
    }
    inline FPS& normalize() {
        while(!this->empty() && this->back() == 0) this-
>pop_back();
        return *this;
    }

    // basic operator
    inline FPS operator-() const noexcept {
        FPS res = (*this);
        for(int i = 0; i < (int)res.size(); ++i) res[i] = -
res[i];
        return res;
    }
    inline FPS operator+(const mint& v) const { return
FPS(*this) += v; }
    inline FPS operator+(const FPS& r) const { return FPS(*this)
+= r; }
    inline FPS operator-(const mint& v) const { return
FPS(*this) -= v; }
    inline FPS operator-(const FPS& r) const { return FPS(*this)
-= r; }
    inline FPS operator*(const mint& v) const { return
FPS(*this) *= v; }
```

```
   inline FPS operator*(const FPS& r) const { return FPS(*this)
*= r; }
   inline FPS operator/(const mint& v) const { return
FPS(*this) /= v; }
   inline FPS operator<<(int x) const { return FPS(*this) <<=
x; }
   inline FPS operator>>(int x) const { return FPS(*this) >>=
x; }
   inline FPS& operator+=(const mint& v) {
      if(this->empty()) this->resize(1);
      (*this)[0] += v;
      return *this;
   }
   inline FPS& operator+=(const FPS& r) {
      if(r.size() > this->size()) this->resize(r.size());
      for(int i = 0; i < (int)r.size(); ++i) (*this)[i] +=
r[i];
      return this->normalize();
   }
   inline FPS& operator-=(const mint& v) {
      if(this->empty()) this->resize(1);
      (*this)[0] -= v;
      return *this;
   }
   inline FPS& operator-=(const FPS& r) {
      if(r.size() > this->size()) this->resize(r.size());
      for(int i = 0; i < (int)r.size(); ++i) (*this)[i] -=
r[i];
      return this->normalize();
   }
   inline FPS& operator*=(const mint& v) {
      for(int i = 0; i < (int)this->size(); ++i) (*this)[i] *=
v;
      return *this;
   }
   inline FPS& operator*=(const FPS& r) { return *this =
NTT::mul((*this), r); }
   inline FPS& operator/=(const mint& v) {
      assert(v != 0);
      mint iv = modinv(v);
      for(int i = 0; i < (int)this->size(); ++i) (*this)[i] *=
iv;
      return *this;
   }
   inline FPS& operator<<=(int x) {
      FPS res(x, 0);
      res.insert(res.end(), begin(*this), end(*this));
      return *this = res;
   }
   inline FPS& operator>>=(int x) {
      FPS res;
      res.insert(res.end(), begin(*this) + x, end(*this));
      return *this = res;
   }
   inline mint eval(const mint& v) {
      mint res = 0;
      for(int i = (int)this->size() - 1; i >= 0; --i) {
         res *= v;
         res += (*this)[i];
      }
      return res;
   }
   inline friend FPS gcd(const FPS& f, const FPS& g) {
      if(g.empty()) return f;
      return gcd(g, f % g);
   }

   // advanced operation
   // df/dx
   inline friend FPS diff(const FPS& f) {
      int n = (int)f.size();
      FPS res(n - 1);
      for(int i = 1; i < n; ++i) res[i - 1] = f[i] * i;
      return res;
   }

   // \int f dx
   inline friend FPS integral(const FPS& f) {
      int n = (int)f.size();
      FPS res(n + 1, 0);
      for(int i = 0; i < n; ++i) res[i + 1] = f[i] / (i + 1);
      return res;
   }

   // inv(f), f[0] must not be 0
   inline friend FPS inv(const FPS& f, int deg) {
      assert(f[0] != 0);
      if(deg < 0) deg = (int)f.size();
      FPS res({mint(1) / f[0]});
      for(int i = 1; i < deg; i <<= 1) { res = (res + res - res
* res * f.pre(i << 1)).pre(i << 1); }
      res.resize(deg);
      return res;
   }
   inline friend FPS inv(const FPS& f) { return inv(f,
f.size()); }

   // division, r must be normalized (r.back() must not be 0)
   inline FPS& operator/=(const FPS& r) {
      assert(!r.empty());
      assert(r.back() != 0);
      this->normalize();
      if(this->size() < r.size()) {
         this->clear();
         return *this;
      }
      int need = (int)this->size() - (int)r.size() + 1;
      *this = ((*this).rev().pre(need) * inv(r.rev(),
need)).pre(need).rev();
      return *this;
   }
   inline FPS& operator%=(const FPS& r) {
      assert(!r.empty());
      assert(r.back() != 0);
      this->normalize();
      FPS q = (*this) / r;
      return *this -= q * r;
   }
   inline FPS operator/(const FPS& r) const { return FPS(*this)
/= r; }
   inline FPS operator%(const FPS& r) const { return FPS(*this)
%= r; }

   // log(f) = \int f'/f dx, f[0] must be 1
   inline friend FPS log(const FPS& f, int deg) {
      assert(f[0] == 1);
      FPS res = integral(diff(f) * inv(f, deg));
      res.resize(deg);
      return res;
   }
   inline friend FPS log(const FPS& f) { return log(f,
f.size()); }

   // exp(f), f[0] must be 0
   inline friend FPS exp(const FPS& f, int deg) {
      assert(f[0] == 0);
      FPS res(1, 1);
      for(int i = 1; i < deg; i <<= 1) { res = res * (f.pre(i
<< 1) - log(res, i << 1) + 1).pre(i << 1); }
      res.resize(deg);
      return res;
   }
   inline friend FPS exp(const FPS& f) { return exp(f,
f.size()); }

   // pow(f) = exp(e * log f)
   inline friend FPS pow(const FPS& f, long long e, int deg) {
      long long i = 0;
      while(i < (int)f.size() && f[i] == 0) ++i;
      if(i == (int)f.size()) return FPS(deg, 0);
      if(i * e >= deg) return FPS(deg, 0);
      mint k = f[i];
      FPS res = exp(log((f >> i) / k, deg) * e, deg) *
modpow(k, e) << (e * i);
      res.resize(deg);
      return res;
   }
   inline friend FPS pow(const FPS& f, long long e) { return
pow(f, e, f.size()); }

   // sqrt(f), f[0] must be 1
   inline friend FPS sqrt_base(const FPS& f, int deg) {
      assert(f[0] == 1);
      mint inv2 = mint(1) / 2;
      FPS res(1, 1);
      for(int i = 1; i < deg; i <<= 1) {
         res = (res + f.pre(i << 1) * inv(res, i << 1)).pre(i
<< 1);
         for(mint& x : res) x *= inv2;
      }
      res.resize(deg);
      return res;
```

```
   }
   inline friend FPS sqrt_base(const FPS& f) { return
sqrt_base(f, f.size()); }
};

// Bostan-Mori
// find [x^N] P(x)/Q(x), O(K log K log N)
// deg(Q(x)) = K, deg(P(x)) < K, Q[0] = 1
template<typename mint> mint BostanMori(const FPS<mint>& P,
const FPS<mint>& Q, long long N) {
   assert(!P.empty() && !Q.empty());
   if(N == 0) return P[0] / Q[0];

   int qdeg = (int)Q.size();
   FPS<mint> P2{P}, minusQ{Q};
   P2.resize(qdeg - 1);
   for(int i = 1; i < (int)Q.size(); i += 2) minusQ[i] = -
minusQ[i];
   P2 *= minusQ;
   FPS<mint> Q2 = Q * minusQ;
   FPS<mint> S(qdeg - 1), T(qdeg);
   for(int i = 0; i < (int)S.size(); ++i) { S[i] = (N % 2 == 0
? P2[i * 2] : P2[i * 2 + 1]); }
   for(int i = 0; i < (int)T.size(); ++i) { T[i] = Q2[i * 2]; }
   return BostanMori(S, T, N >> 1);
}
```

dynamic_convex_hull_trick.hpp

md5: 08b8ef

```
template<typename T> class CHT {
   private:
   struct node {
      node *left, *right;
      static const T inf = numeric_limits<T>::max();
      T a, b, l, r;
      node(const T _a, const T _b, const T _l, const T _r)
         : left(nullptr), right(nullptr), a(_a), b(_b), l(_l),
r(_r) {}
      T f(const T x) const { return (l <= x && x < r) ? (a * x
+ b) : inf; }
      T tf(const T x) const { return a * x + b; }
      bool isLine(const T _l, const T _r) const { return (l ==
_l) && (r == _r); }
   };
   static void swap(node* x, node* y) {
      std::swap(x->a, y->a), std::swap(x->b, y->b),
std::swap(x->l, y->l), std::swap(x->r, y->r);
   }
   void _add_segment(node* cur, node* nw, T l, T r) {
      while(true) {
         const T mid = (l + r) / 2;
         if(cur->f(nw->l) <= nw->tf(nw->l) && cur->f(nw->r - 1)
<= nw->tf(nw->r - 1)) { break; }
         if(cur->tf(cur->l) >= nw->f(cur->l) && cur->tf(cur->r
- 1) >= nw->f(cur->r - 1)) {
            swap(cur, nw);
            break;
         }
         if(cur->isLine(l, r) && nw->isLine(l, r)) {
            if(nw->tf(l) < cur->tf(l)) swap(cur, nw);
            if(cur->tf(mid) <= nw->tf(mid)) {
               if(!cur->right) {
                  cur->right = new node(nw->a, nw->b, mid, r);
                  break;
               } else {
                  cur = cur->right, l = mid, nw->l = mid;
               }
            } else {
               swap(cur, nw);
               if(!cur->left) {
                  cur->left = new node(nw->a, nw->b, l, mid);
                  break;
               } else {
                  cur = cur->left, r = mid, nw->r = mid;
               }
            }
            continue;
         }
         if(nw->isLine(l, r)) swap(cur, nw);
         if(nw->r <= mid) {
            if(!cur->left) {
               cur->left = new node(*nw);
               break;
            } else {
               cur = cur->left, r = mid;
            }
         } else if(mid <= nw->l) {
            if(!cur->right) {
               cur->right = new node(*nw);
               break;
            } else {
               cur = cur->right, l = mid;
            }
         } else {
            node _nw(nw->a, nw->b, mid, nw->r);
            nw->r = mid;
            if(!cur->left) {
               cur->left = new node(*nw);
            } else {
               _add_segment(cur->left, nw, l, mid);
            }
            if(!cur->right) {
               cur->right = new node(_nw);
            } else {
               _add_segment(cur->right, &_nw, mid, r);
            }
            break;
         }
      }
   }
   T query(node* cur, const T k, T l, T r) const {
      T ans = numeric_limits<T>::max();
      while(cur) {
         ans = min(ans, cur->f(k));
         const T mid = (l + r) / 2;
         if(k < mid) {
            cur = cur->left, r = mid;
         } else {
            cur = cur->right, l = mid;
         }
      }
      return ans;
   }
   void clear(node* cur) {
      if(cur->left) clear(cur->left);
      if(cur->right) clear(cur->right);
      delete cur;
   }
   const T lpos, rpos;
   node* root;

   public:
   CHT(const T _lpos, const T _rpos)
       : lpos(_lpos), rpos(_rpos), root(new node(0,
numeric_limits<T>::max(), lpos, rpos)) {
      assert(lpos < rpos);
   }
   // ~CHT(){ clear(root); }
   // f(x) = a * x + b を挿入
   void add_line(const T a, const T b) {
      node nw(a, b, lpos, rpos);
      return _add_line(root, &nw, lpos, rpos);
   }
   // f(x) = a * x + b (x ∈ [l, r)) を挿入
   void add_segment(const T a, const T b, const T l, const T r)
{
      assert(l < r);
      node nw(a, b, l, r);
      return _add_segment(root, &nw, lpos, rpos);
   }
   // x = k での最小値
   T query(const T k) const { return query(root, k, lpos,
rpos); }
};
/*使用例
void solve(int test){
    int n,q; cin >> n >> q;
    CHT<ll> cht(-1000000000,1000000000);
    rep(i,n){
        ll l,r,a,b; cin >> l >> r >> a >> b;
        cht.add_segment(a,b,l,r);
    }
    rep(qi,q){
        int t; cin >> t;
        if(t==0){
            ll l,r,a,b; cin >> l >> r >> a >> b;
            cht.add_segment(a,b,l,r);
        }
        else {
            ll p; cin >> p;
            ll res=cht.query(p);
```

```
            if(res == numeric_limits<ll>::max()){
                cout << "INFINITY" << endl;
            }else{
                cout << res << endl;
            }
        }
    }
}
```

dynamic_segtree.hpp

md5: 279c02

```
template<typename T> class dynamic_segtree {
   private:
   struct node {
      T val, lazy;
      int st_size;
      node *par, *left, *right;
      node() : val(id2), lazy(id1), st_size(0), par(nullptr),
left(nullptr), right(nullptr) {}
   };
   void opr1(T& arg1, T arg2) { arg1 += arg2; }
   T opr2(T arg1, T arg2) { return arg1 + arg2; }
   int nw_size;
   static const int POOL_SIZE = 4000000;
   static const int MAX_SIZE = 100001;
   static node* root;
   static node pool[POOL_SIZE];
   static const T id1 = (T)0;
   static const T id2 = (T)0;
   node* alloc() {
      assert(nw_size < POOL_SIZE);
      return (&pool[nw_size++]);
   }
   void eval(node* k, int l, int r) {
      if(k->lazy != id1) {
         opr1(k->val, k->lazy);
         if(r - l > 1) {
            if(k->left) { opr1(k->left->lazy, k->lazy / k-
>st_size * k->left->st_size); }
            if(k->right) { opr1(k->right->lazy, k->lazy / k-
>st_size * k->right->st_size); }
         }
         k->lazy = id1;
      }
   }

   public:
   dynamic_segtree() : nw_size(0) { root = alloc(); }
   // a 番目に x を追加(すでに a 番目に値が存在すればそれに x を足す)
   void insert(int a, T x, node* k = root, int l = 0, int r =
MAX_SIZE) {
      eval(k, l, r);
      k->st_size++;
      k->val = opr2(k->val, x);
      if(r - l == 1) { return; }
      if(a < (l + r) / 2) {
         if(!(k->left)) {
            k->left = alloc();
            k->left->par = k;
         }
         insert(a, x, k->left, l, (l + r) / 2);
      } else {
         if(!(k->right)) {
            k->right = alloc();
            k->right->par = k;
         }
         insert(a, x, k->right, (l + r) / 2, r);
      }
   }
   void range(int a, int b, const T x, node* k = root, int l =
0, int r = MAX_SIZE) {
      eval(k, l, r);
      if(b <= l || r <= a) return;
      if(a <= l && r <= b) {
         opr1(k->lazy, k->st_size * x);
         eval(k, l, r);
         return;
      }
      k->val = id2;
      if(k->left) {
         range(a, b, x, k->left, l, (l + r) / 2);
         k->val = opr2(k->val, k->left->val);
      }
      if(k->right) {
         range(a, b, x, k->right, (l + r) / 2, r);
         k->val = opr2(k->val, k->right->val);
      }
   }
   T query(int a, int b, node* k = root, int l = 0, int r =
MAX_SIZE) {
      eval(k, l, r);
      if(b <= l || r <= a) { return id2; }
      if(a <= l && r <= b) { return k->val; }
      T vl = id2, vr = id2;
      if(k->left) { vl = query(a, b, k->left, l, (l + r) / 2);
}
      if(k->right) { vr = query(a, b, k->right, (l + r) / 2,
r); }
      return opr2(vl, vr);
   }
};
template<typename T> class dynamic_segtree<T>::node*
dynamic_segtree<T>::root;
template<typename T> class dynamic_segtree<T>::node
dynamic_segtree<T>::pool[POOL_SIZE];
```

fps.hpp

md5: 9d0fc3

```
#include <atcoder/all>
#include <iostream>   // cout, endl, cin
#include <string>     // string, to_string, stoi
#include <vector>     // vector
#include <algorithm>  // min, max, swap, sort, reverse,
lower_bound, upper_bound
#include <utility>    // pair, make_pair
#include <tuple>      // tuple, make_tuple
#include <cstdint>    // int64_t, int*_t
#include <cstdio>     // printf
#include <map>        // map
#include <queue>      // queue, priority_queue
#include <set>        // set
#include <stack>      // stack
#include <deque>  // deque #include <unordered_map> //
unordered_map #include <unordered_set> // unordered_set
#include <bitset> // bitset
#include <cctype>  // isupper, islower, isdigit, toupper,
tolower
#include <math.h>
#include <iomanip>
#include <functional>
#include <unordered_map>
#include <random>
#include <bitset>
#include <cassert>
using namespace std;
using namespace atcoder;
#define rep(i, n) for(int i = 0; i < (int)(n); i++)
#define repi(i, a, b) for(int i = (int)(a); i < (int)(b); i++)
#define endl '\n'
#define all(x) (x).begin(), (x).end()
#define arr(x) (x).rbegin(), (x).rend()
typedef long long ll;
typedef unsigned long long ull;
const ll inf = 4e18;
using graph = vector<vector<int> >;
using vi = vector<int>;
using P = pair<ll, ll>;
using vvi = vector<vi>;
using vvvi = vector<vvi>;
using vll = vector<ll>;
using vvll = vector<vll>;
using vp = vector<P>;
using vvp = vector<vp>;
using vd = vector<double>;
using vvd = vector<vd>;
// using vs=vector<string>;
// string T="ABCDEFGHIJKLMNOPQRSTUVWXYZ";
// string S="abcdefghijklmnopqrstuvwxyz";
// g++ main.cpp -std=c++17 -I .
// cout <<setprecision(20);
// cout << fixed << setprecision(10);
// cin.tie(0); ios::sync_with_stdio(false);

const double PI = acos(-1);
int vx[] = {0, 1, 0, -1, -1, 1, 1, -1}, vy[] = {1, 0, -1, 0, 1,
1, -1, -1};
void putsYes(bool f) { cout << (f ? "Yes" : "No") << endl; }
void putsYES(bool f) { cout << (f ? "YES" : "NO") << endl; }
void putsFirst(bool f) { cout << (f ? "First" : "Second") <<
endl; }
```

```
void debug(int test) { cout << "TEST" << " " << test << endl; }
ll pow_pow(ll x, ll n, ll mod) {
   if(n == 0) return 1;
   x %= mod;
   ll res = pow_pow(x * x % mod, n / 2, mod);
   if(n & 1) res = res * x % mod;
   return res;
}
ll gcdll(ll x, ll y) {
   if(y == 0) return x;
   return gcdll(y, x % y);
}
ll INFF = 1000100100100100100;
ll lcmll(ll x, ll y) {
   ll X = (x / gcdll(x, y)), Y = y;
   if(X <= INFF / Y) return X * Y;
   else return INFF;
}
template<class T> bool chmin(T& a, T b) {
   if(a > b) {
      a = b;
      return true;
   } else return false;
}
template<class T> bool chmax(T& a, T b) {
   if(a < b) {
      a = b;
      return true;
   } else return false;
}
struct edge {
   int to;
   ll cost;
   int id;
   edge(int to, ll cost, int id) : to(to), cost(cost), id(id)
{}
};
using ve = vector<edge>;
using vve = vector<ve>;
template<class T> vector<T> NTT(vector<T> a, vector<T> b) {
   ll nmod = T::mod();
   int n = a.size();
   int m = b.size();
   vector<int> x1(n);
   vector<int> y1(m);
   for(int i = 0; i < n; i++) {
      ll tmp1, tmp2, tmp3;
      tmp1 = a[i].val();
      x1[i] = tmp1;
   }
   for(int i = 0; i < m; i++) {
      ll tmp1, tmp2, tmp3;
      tmp1 = b[i].val();
      y1[i] = tmp1;
   }
   auto z1 = convolution<167772161>(x1, y1);
   auto z2 = convolution<469762049>(x1, y1);
   auto z3 = convolution<1224736769>(x1, y1);
   vector<T> res(n + m - 1);
   constexpr ll m1 = 167772161;
   constexpr ll m2 = 469762049;
   constexpr ll m3 = 1224736769;
   constexpr ll m1m2 = 104391568;
   constexpr ll m1m2m3 = 721017874;
   ll mm12 = m1 * m2 % nmod;
   for(int i = 0; i < n + m - 1; i++) {
      int v1 = (z2[i] - z1[i]) * m1m2 % m2;
      if(v1 < 0) v1 += m2;
      int v2 = (z3[i] - (z1[i] + v1 * m1) % m3) * m1m2m3 % m3;
      if(v2 < 0) v2 += m3;
      res[i] = (z1[i] + v1 * m1 + v2 * mm12);
   }
   return res;
}
template<class T> struct FormalPowerSeries : vector<T> {
   using vector<T>::vector;
   using F = FormalPowerSeries;
   F& operator=(const vector<T>& g) {
      int n = g.size();
      int m = (*this).size();
      if(m < n) (*this).resize(n);
      for(int i = 0; i < n; i++) (*this)[i] = g[i];
      return (*this);
   }
   F& operator=(const F& g) {
      int n = g.size();
      int m = (*this).size();
      if(m < n) (*this).resize(n);
      for(int i = 0; i < n; i++) (*this)[i] = g[i];
      return (*this);
   }
   F& operator-() {
      for(int i = 0; i < (*this).size(); i++) (*this)[i] *= -1;
      return (*this);
   }
   F& operator+=(const F& g) {
      int n = (*this).size();
      int m = g.size();
      if(n < m) (*this).resize(m);
      for(int i = 0; i < m; i++) (*this)[i] += g[i];
      return (*this);
   }
   F& operator+=(const T& r) {
      if((*this).size() == 0) (*this).resize(1);
      (*this)[0] += r;
      return (*this);
   }
   F& operator-=(const F& g) {
      int n = (*this).size();
      int m = g.size();
      if(n < m) (*this).resize(m);
      for(int i = 0; i < m; i++) (*this)[i] -= g[i];
      return (*this);
   }
   F& operator-=(const T& r) {
      if((*this).size() == 0) (*this).resize(1);
      (*this)[0] -= r;
      return (*this);
   }
   F& operator*=(const F& g) {
      (*this) = convolution((*this), g);
      return (*this);
   }
   F& operator*=(const T& r) {
      for(int i = 0; i < (*this).size(); i++) (*this)[i] *= r;
      return (*this);
   }
   F& operator/=(const F& g) {
      int n = (*this).size();
      (*this) = convolution((*this), g.inv());
      (*this).resize(n);
      return (*this);
   }
   F& operator/=(const T& r) {
      r = r.inv();
      for(int i = 0; i < (*this).size(); i++) (*this)[i] *= r;
      return (*this);
   }
   F& operator<<=(const int d) {
      int n = (*this).size();
      (*this).insert((*this).begin(), d, 0);
      (*this).resize(n);
      return *this;
   }
   F& operator>>=(const int d) {
      int n = (*this).size();
      (*this).erase((*this).begin(), (*this).begin() + min(n,
d));
      (*this).resize(n);
      return *this;
   }
   F operator*(const T& g) const { return F(*this) *= g; }
   F operator-(const T& g) const { return F(*this) -= g; }
   F operator+(const T& g) const { return F(*this) += g; }
   F operator/(const T& g) const { return F(*this) /= g; }
   F operator*(const F& g) const { return F(*this) *= g; }
   F operator-(const F& g) const { return F(*this) -= g; }
   F operator+(const F& g) const { return F(*this) += g; }
   F operator/(const F& g) const { return F(*this) /= g; }
   F operator%(const F& g) const { return F(*this) %= g; }
   F operator<<(const int d) const { return F(*this) <<= d; }
   F operator>>(const int d) const { return F(*this) >>= d; }
   F pre(int sz) const { return F(begin(*this), begin(*this) +
min((int)this->size(), sz)); }
   F inv(int deg = -1) const {
      int n = (*this).size();
      if(deg == -1) deg = n;
      assert(n > 0 && (*this)[0] != T(0));
      F g(1);
      g[0] = (*this)[0].inv();
      while(g.size() < deg) {
         int m = g.size();
```

```
        F f(begin(*this), begin(*this) + min(n, 2 * m));
        F r(g);
        f.resize(2 * m);
        r.resize(2 * m);
        internal::butterfly(f);
        internal::butterfly(r);
        for(int i = 0; i < 2 * m; i++) f[i] *= r[i];
        internal::butterfly_inv(f);
        f.erase(f.begin(), f.begin() + m);
        f.resize(2 * m);
        internal::butterfly(f);
        for(int i = 0; i < 2 * m; i++) f[i] *= r[i];
        internal::butterfly_inv(f);
        T in = T(2 * m).inv();
        in *= -in;
        for(int i = 0; i < m; i++) f[i] *= in;
        g.insert(g.end(), f.begin(), f.begin() + m);
      }
      return g.pre(deg);
   }
   T eval(const T& a) {
      T x = 1;
      T ret = 0;
      for(int i = 0; i < (*this).size(); i++) {
         ret += (*this)[i] * x;
         x *= a;
      }
      return ret;
   }
   void onemul(const int d, const T c) {
      int n = (*this).size();
      for(int i = n - d - 1; i >= 0; i--) { (*this)[i + d] +=
(*this)[i] * c; }
   }
   void onediv(const int d, const T c) {
      int n = (*this).size();
      for(int i = 0; i < n - d; i++) { (*this)[i + d] -=
(*this)[i] * c; }
   }
   F diff() const {
      int n = (*this).size();
      F ret(n);
      for(int i = 1; i < n; i++) ret[i - 1] = (*this)[i] * i;
      ret[n - 1] = 0;
      return ret;
   }
   F integral() const {
      int n = (*this).size(), mod = T::mod();
      vector<T> inv(n);
      inv[1] = 1;
      for(int i = 2; i < n; i++) inv[i] = T(mod) - inv[mod % i]
* (mod / i);
      F ret(n);
      for(int i = n - 2; i >= 0; i--) ret[i + 1] = (*this)[i] *
inv[i + 1];
      ret[0] = 0;
      return ret;
   }
   F log(int deg = -1) const {
      int n = (*this).size();
      if(deg == -1) deg = n;
      assert((*this)[0] == T(1));
      return ((*this).diff() *
(*this).inv(deg)).pre(deg).integral();
   }
   F exp(int deg = -1) const {
      int n = (*this).size();
      if(deg == -1) deg = n;
      assert(n == 0 || (*this)[0] == 0);
      F Inv;
      Inv.reserve(deg);
      Inv.push_back(T(0));
      Inv.push_back(T(1));
      auto inplace_integral = [&](F& f) -> void {
         const int n = (int)f.size();
         int mod = T::mod();
         while(Inv.size() <= n) {
            int i = Inv.size();
            Inv.push_back((-Inv[mod % i]) * (mod / i));
         }
         f.insert(begin(f), T(0));
         for(int i = 1; i <= n; i++) f[i] *= Inv[i];
      };
      auto inplace_diff = [](F& f) -> void {
         if(f.empty()) return;
         f.erase(begin(f));
         T coeff = 1, one = 1;
         for(int i = 0; i < f.size(); i++) {
            f[i] *= coeff;
            coeff++;
         }
      };
      F b{1, 1 < (int)(*this).size() ? (*this)[1] : 0}, c{1},
z1, z2{1, 1};
      for(int m = 2; m <= deg; m <<= 1) {
         auto y = b;
         y.resize(2 * m);
         internal::butterfly(y);
         z1 = z2;
         F z(m);
         for(int i = 0; i < m; i++) z[i] = y[i] * z1[i];
         internal::butterfly_inv(z);
         T si = T(m).inv();
         for(int i = 0; i < m; i++) z[i] *= si;
         fill(begin(z), begin(z) + m / 2, T(0));
         internal::butterfly(z);
         for(int i = 0; i < m; i++) z[i] *= -z1[i];
         internal::butterfly_inv(z);
         for(int i = 0; i < m; i++) z[i] *= si;
         c.insert(end(c), begin(z) + m / 2, end(z));
         z2 = c;
         z2.resize(2 * m);
         internal::butterfly(z2);
         F x(begin((*this)), begin((*this)) + min<int>
((*this).size(), m));
         x.resize(m);
         inplace_diff(x);
         x.push_back(T(0));
         internal::butterfly(x);
         for(int i = 0; i < m; i++) x[i] *= y[i];
         internal::butterfly_inv(x);
         for(int i = 0; i < m; i++) x[i] *= si;
         x -= b.diff();
         x.resize(2 * m);
         for(int i = 0; i < m - 1; i++) x[m + i] = x[i], x[i] =
T(0);
         internal::butterfly(x);
         for(int i = 0; i < 2 * m; i++) x[i] *= z2[i];
         internal::butterfly_inv(x);
         T si2 = T(m << 1).inv();
         for(int i = 0; i < 2 * m; i++) x[i] *= si2;
         x.pop_back();
         inplace_integral(x);
         for(int i = m; i < min<int>((*this).size(), 2 * m);
i++) x[i] += (*this)[i];
         fill(begin(x), begin(x) + m, T(0));
         internal::butterfly(x);
         for(int i = 0; i < 2 * m; i++) x[i] *= y[i];
         internal::butterfly_inv(x);
         for(int i = 0; i < 2 * m; i++) x[i] *= si2;
         b.insert(end(b), begin(x) + m, end(x));
      }
      return b.pre(deg);
   }
   F pow(ll m) {
      int n = (*this).size();
      int x = 0;
      while(x < (*this).size() && (*this)[x] == T(0)) { x++; }
      if(x * m >= n) {
         F ret(n);
         return ret;
      }
      F f(n - x);
      T y = (*this)[x];
      for(int i = x; i < n; i++) f[i - x] = (*this)[i] / y;
      f = f.log();
      for(int i = 0; i < f.size(); i++) f[i] *= m;
      f = f.exp();
      y = y.pow(m);
      for(int i = 0; i < f.size(); i++) f[i] *= y;
      F ret(n);
      for(int i = x * m; i < n; i++) ret[i] = f[i - x * m];
      return ret;
   }
   F shift(T c) {
      int n = (*this).size();
      int mod = T::mod();
      vector<T> inv(n + 1);
      inv[1] = 1;
      for(int i = 2; i <= n; i++) inv[i] = mod - inv[mod % i] *
(mod / i);
      T x = 1;
```

```
      for(int i = 0; i < n; i++) {
         (*this)[i] *= x;
         x *= (i + 1);
      }
      F g(n);
      T y = 1;
      T now = 1;
      for(int i = 0; i < n; i++) {
         g[n - i - 1] = now * y;
         now *= c;
         y *= inv[i + 1];
      }
      auto tmp = convolution(g, (*this));
      T z = 1;
      for(int i = 0; i < n; i++) {
         (*this)[i] = tmp[n + i - 1] * z;
         z *= inv[i + 1];
      }
      return (*this);
   }
};
using mint = modint998244353;
using fps = FormalPowerSeries<mint>;
const int mod = 998244353;
// using mint = ;
// using vm=vector<mint>;
// using vvm=vector<vm>;
// using vvvm=vector<vvm>;
constexpr int MAX = 10;
ll fact[MAX], finv[MAX], inv[MAX];
void initcomb() {
   fact[0] = fact[1] = 1;
   finv[0] = finv[1] = 1;
   inv[1] = 1;
   for(int i = 2; i < MAX; i++) {
      fact[i] = fact[i - 1] * i % mod;
      inv[i] = mod - inv[mod % i] * (mod / i) % mod;
      finv[i] = finv[i - 1] * inv[i] % mod;
   }
}
ll comb(ll n, ll k) {
   if(n < k) return 0;
   if(n < 0 || k < 0) return 0;
   return fact[n] * (finv[k] * finv[n - k] % mod) % mod;
}
vll anss;
void solve(int test) {
   int n;
   cin >> n;
   fps f(n);
   rep(i, n) f[i] = 1;
   f = f.pow(2);
   rep(i, n) cout << f[i].val() << " ";
   cout << endl;
}
// g++ cf.cpp -std=c++17 -I .
int main() {
   cin.tie(0);
   ios::sync_with_stdio(false);
   { initcomb(); }
   int t = 1;  // cin >>t;
   rep(test, t) solve(test);
   for(auto ans : anss) { cout << ans << endl; }
}
```

geometry.hpp

md5: 49dca1

```
using Real = double;
using Point = complex<Real>;
const Real EPS = 1e-8, PI = acos(-1);

inline bool eq(Real a, Real b) { return fabs(b - a) < EPS; }

Point operator*(const Point& p, const Real& d) { return
Point(real(p) * d, imag(p) * d); }

istream& operator>>(istream& is, Point& p) {
   Real a, b;
   is >> a >> b;
   p = Point(a, b);
   return is;
}

ostream& operator<<(ostream& os, Point& p) { return os << fixed
<< setprecision(10) << p.real() << " " << p.imag(); }

// 点 p を反時計回りに theta 回転
Point rotate(Real theta, const Point& p) {
   return Point(cos(theta) * p.real() - sin(theta) * p.imag(),
sin(theta) * p.real() + cos(theta) * p.imag());
}

Real radian_to_degree(Real r) { return (r * 180.0 / PI); }

Real degree_to_radian(Real d) { return (d * PI / 180.0); }

// a-b-c の角度のうち小さい方を返す
Real get_angle(const Point& a, const Point& b, const Point& c)
{
   const Point v(b - a), w(c - b);
   Real alpha = atan2(v.imag(), v.real()), beta =
atan2(w.imag(), w.real());
   if(alpha > beta) swap(alpha, beta);
   Real theta = (beta - alpha);
   return min(theta, 2 * acos(-1) - theta);
}

namespace std {
bool operator<(const Point& a, const Point& b) {
   return a.real() != b.real() ? a.real() < b.real() : a.imag()
< b.imag();
}
}  // namespace std

struct Line {
   Point a, b;

   Line() = default;

   Line(Point a, Point b) : a(a), b(b) {}

   Line(Real A, Real B, Real C)  // Ax + By = C
   {
      if(eq(A, 0)) a = Point(0, C / B), b = Point(1, C / B);
      else if(eq(B, 0)) b = Point(C / A, 0), b = Point(C / A,
1);
      else a = Point(0, C / B), b = Point(C / A, 0);
   }

   friend ostream& operator<<(ostream& os, Line& p) { return os
<< p.a << " to " << p.b; }

   friend istream& operator>>(istream& is, Line& a) { return is
>> a.a >> a.b; }
};

struct Segment : Line {
   Segment() = default;

   Segment(Point a, Point b) : Line(a, b) {}
};

struct Circle {
   Point p;
   Real r;

   Circle() = default;

   Circle(Point p, Real r) : p(p), r(r) {}
};

using Points = vector<Point>;
using Polygon = vector<Point>;
using Segments = vector<Segment>;
using Lines = vector<Line>;
using Circles = vector<Circle>;

Real cross(const Point& a, const Point& b) { return real(a) *
imag(b) - imag(a) * real(b); }

Real dot(const Point& a, const Point& b) { return real(a) *
real(b) + imag(a) * imag(b); }

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_1_C
// 点の回転方向
int ccw(const Point& a, Point b, Point c) {
   b = b - a, c = c - a;
   if(cross(b, c) > EPS) return +1;   // "COUNTER_CLOCKWISE"
   if(cross(b, c) < -EPS) return -1;  // "CLOCKWISE"
   if(dot(b, c) < 0) return +2;       // "ONLINE_BACK"
```

```
   if(norm(b) < norm(c)) return -2;   // "ONLINE_FRONT"
   return 0;                          // "ON_SEGMENT"
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_2_A
// 平行判定
bool parallel(const Line& a, const Line& b) { return
eq(cross(a.b - a.a, b.b - b.a), 0.0); }

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_2_A
// 垂直判定
bool orthogonal(const Line& a, const Line& b) { return
eq(dot(a.a - a.b, b.a - b.b), 0.0); }

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_1_A
// 射影
// 直線 l に p から垂線を引いた交点を求める
Point projection(const Line& l, const Point& p) {
   double t = dot(p - l.a, l.a - l.b) / norm(l.a - l.b);
   return l.a + (l.a - l.b) * t;
}

Point projection(const Segment& l, const Point& p) {
   double t = dot(p - l.a, l.a - l.b) / norm(l.a - l.b);
   return l.a + (l.a - l.b) * t;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_1_B
// 反射
// 直線 l を対称軸として点 p  と線対称にある点を求める
Point reflection(const Line& l, const Point& p) { return p +
(projection(l, p) - p) * 2.0; }

bool intersect(const Line& l, const Point& p) { return
abs(ccw(l.a, l.b, p)) != 1; }

bool intersect(const Line& l, const Line& m) {
   return abs(cross(l.b - l.a, m.b - m.a)) > EPS ||
abs(cross(l.b - l.a, m.b - l.a)) < EPS;
}

bool intersect(const Segment& s, const Point& p) { return
ccw(s.a, s.b, p) == 0; }

bool intersect(const Line& l, const Segment& s) {
   return cross(l.b - l.a, s.a - l.a) * cross(l.b - l.a, s.b -
l.a) < EPS;
}

Real distance(const Line& l, const Point& p);

bool intersect(const Circle& c, const Line& l) { return
distance(l, c.p) <= c.r + EPS; }

bool intersect(const Circle& c, const Point& p) { return
abs(abs(p - c.p) - c.r) < EPS; }

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_2_B
bool intersect(const Segment& s, const Segment& t) {
   return ccw(s.a, s.b, t.a) * ccw(s.a, s.b, t.b) <= 0 &&
ccw(t.a, t.b, s.a) * ccw(t.a, t.b, s.b) <= 0;
}

int intersect(const Circle& c, const Segment& l) {
   if(norm(projection(l, c.p) - c.p) - c.r * c.r > EPS) return
0;
   auto d1 = abs(c.p - l.a), d2 = abs(c.p - l.b);
   if(d1 < c.r + EPS && d2 < c.r + EPS) return 0;
   if(d1 < c.r - EPS && d2 > c.r + EPS || d1 > c.r + EPS && d2
< c.r - EPS) return 1;
   const Point h = projection(l, c.p);
   if(dot(l.a - h, l.b - h) < 0) return 2;
   return 0;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_7_A&lang=jp
int intersect(Circle c1, Circle c2) {
   if(c1.r < c2.r) swap(c1, c2);
   Real d = abs(c1.p - c2.p);
   if(c1.r + c2.r < d) return 4;
   if(eq(c1.r + c2.r, d)) return 3;
   if(c1.r - c2.r < d) return 2;
   if(eq(c1.r - c2.r, d)) return 1;
   return 0;
}

Real distance(const Point& a, const Point& b) { return abs(a -
b); }

Real distance(const Line& l, const Point& p) { return abs(p -
projection(l, p)); }

Real distance(const Line& l, const Line& m) { return
intersect(l, m) ? 0 : distance(l, m.a); }

Real distance(const Segment& s, const Point& p) {
   Point r = projection(s, p);
   if(intersect(s, r)) return abs(r - p);
   return min(abs(s.a - p), abs(s.b - p));
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_2_D
Real distance(const Segment& a, const Segment& b) {
   if(intersect(a, b)) return 0;
   return min({distance(a, b.a), distance(a, b.b), distance(b,
a.a), distance(b, a.b)});
}

Real distance(const Line& l, const Segment& s) {
   if(intersect(l, s)) return 0;
   return min(distance(l, s.a), distance(l, s.b));
}

Point crosspoint(const Line& l, const Line& m) {
   Real A = cross(l.b - l.a, m.b - m.a);
   Real B = cross(l.b - l.a, l.b - m.a);
   if(eq(abs(A), 0.0) && eq(abs(B), 0.0)) return m.a;
   return m.a + (m.b - m.a) * B / A;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_2_C
Point crosspoint(const Segment& l, const Segment& m) { return
crosspoint(Line(l), Line(m)); }

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_7_D
pair<Point, Point> crosspoint(const Circle& c, const Line l) {
   Point pr = projection(l, c.p);
   Point e = (l.b - l.a) / abs(l.b - l.a);
   if(eq(distance(l, c.p), c.r)) return {pr, pr};
   double base = sqrt(c.r * c.r - norm(pr - c.p));
   return {pr - e * base, pr + e * base};
}

pair<Point, Point> crosspoint(const Circle& c, const Segment&
l) {
   Line aa = Line(l.a, l.b);
   if(intersect(c, l) == 2) return crosspoint(c, aa);
   auto ret = crosspoint(c, aa);
   if(dot(l.a - ret.first, l.b - ret.first) < 0) ret.second =
ret.first;
   else ret.first = ret.second;
   return ret;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_7_E
pair<Point, Point> crosspoint(const Circle& c1, const Circle&
c2) {
   Real d = abs(c1.p - c2.p);
   Real a = acos((c1.r * c1.r + d * d - c2.r * c2.r) / (2 *
c1.r * d));
   Real t = atan2(c2.p.imag() - c1.p.imag(), c2.p.real() -
c1.p.real());
   Point p1 = c1.p + Point(cos(t + a) * c1.r, sin(t + a) *
c1.r);
   Point p2 = c1.p + Point(cos(t - a) * c1.r, sin(t - a) *
c1.r);
   return {p1, p2};
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_7_F
// 点 p を通る円 c の接線
```

```
pair<Point, Point> tangent(const Circle& c1, const Point& p2) {
   return crosspoint(c1, Circle(p2, sqrt(norm(c1.p - p2) - c1.r
* c1.r)));
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_7_G
// 円 c1, c2 の共通接線
Lines tangent(Circle c1, Circle c2) {
   Lines ret;
   if(c1.r < c2.r) swap(c1, c2);
   Real g = norm(c1.p - c2.p);
   if(eq(g, 0)) return ret;
   Point u = (c2.p - c1.p) / sqrt(g);
   Point v = rotate(PI * 0.5, u);
   for(int s : {-1, 1}) {
      Real h = (c1.r + s * c2.r) / sqrt(g);
      if(eq(1 - h * h, 0)) {
         ret.emplace_back(c1.p + u * c1.r, c1.p + (u + v) *
c1.r);
      } else if(1 - h * h > 0) {
         Point uu = u * h, vv = v * sqrt(1 - h * h);
         ret.emplace_back(c1.p + (uu + vv) * c1.r, c2.p - (uu +
vv) * c2.r * s);
         ret.emplace_back(c1.p + (uu - vv) * c1.r, c2.p - (uu -
vv) * c2.r * s);
      }
   }
   return ret;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_3_B
// 凸性判定
bool is_convex(const Polygon& p) {
   int n = (int)p.size();
   for(int i = 0; i < n; i++) {
      if(ccw(p[(i + n - 1) % n], p[i], p[(i + 1) % n]) == -1)
return false;
   }
   return true;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_4_A
// 凸包
Polygon convex_hull(Polygon& p) {
   int n = (int)p.size(), k = 0;
   if(n <= 2) return p;
   sort(p.begin(), p.end());
   vector<Point> ch(2 * n);
   for(int i = 0; i < n; ch[k++] = p[i++]) {
      while(k >= 2 && cross(ch[k - 1] - ch[k - 2], p[i] - ch[k
- 1]) < EPS) --k;
   }
   for(int i = n - 2, t = k + 1; i >= 0; ch[k++] = p[i--]) {
      while(k >= t && cross(ch[k - 1] - ch[k - 2], p[i] - ch[k
- 1]) < EPS) --k;
   }
   ch.resize(k - 1);
   return ch;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_3_C
// 多角形と点の包含判定
enum { OUT, ON, IN };

int contains(const Polygon& Q, const Point& p) {
   bool in = false;
   for(int i = 0; i < Q.size(); i++) {
      Point a = Q[i] - p, b = Q[(i + 1) % Q.size()] - p;
      if(a.imag() > b.imag()) swap(a, b);
      if(a.imag() <= 0 && 0 < b.imag() && cross(a, b) < 0) in =
!in;
      if(cross(a, b) == 0 && dot(a, b) <= 0) return ON;
   }
   return in ? IN : OUT;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=1033
// 線分の重複除去
void merge_segments(vector<Segment>& segs) {
   auto merge_if_able = [](Segment& s1, const Segment& s2) {
      if(abs(cross(s1.b - s1.a, s2.b - s2.a)) > EPS) return
false;
      if(ccw(s1.a, s2.a, s1.b) == 1 || ccw(s1.a, s2.a, s1.b) ==
-1) return false;
      if(ccw(s1.a, s1.b, s2.a) == -2 || ccw(s2.a, s2.b, s1.a)
== -2) return false;
      s1 = Segment(min(s1.a, s2.a), max(s1.b, s2.b));
      return true;
   };

   for(int i = 0; i < segs.size(); i++) {
      if(segs[i].b < segs[i].a) swap(segs[i].a, segs[i].b);
   }
   for(int i = 0; i < segs.size(); i++) {
      for(int j = i + 1; j < segs.size(); j++) {
         if(merge_if_able(segs[i], segs[j])) { segs[j--] =
segs.back(), segs.pop_back(); }
      }
   }
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=1033
// 線分アレンジメント
// 任意の2線分の交点を頂点としたグラフを構築する
vector<vector<int> > segment_arrangement(vector<Segment>& segs,
vector<Point>& ps) {
   vector<vector<int> > g;
   int N = (int)segs.size();
   for(int i = 0; i < N; i++) {
      ps.emplace_back(segs[i].a);
      ps.emplace_back(segs[i].b);
      for(int j = i + 1; j < N; j++) {
         const Point p1 = segs[i].b - segs[i].a;
         const Point p2 = segs[j].b - segs[j].a;
         if(cross(p1, p2) == 0) continue;
         if(intersect(segs[i], segs[j])) {
ps.emplace_back(crosspoint(segs[i], segs[j])); }
      }
   }
   sort(begin(ps), end(ps));
   ps.erase(unique(begin(ps), end(ps)), end(ps));

   int M = (int)ps.size();
   g.resize(M);
   for(int i = 0; i < N; i++) {
      vector<int> vec;
      for(int j = 0; j < M; j++) {
         if(intersect(segs[i], ps[j])) { vec.emplace_back(j); }
      }
      for(int j = 1; j < vec.size(); j++) {
         g[vec[j - 1]].push_back(vec[j]);
         g[vec[j]].push_back(vec[j - 1]);
      }
   }
   return (g);
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_4_C
// 凸多角形の切断
// 直線 l.a-l.b で切断しその左側にできる凸多角形を返す
Polygon convex_cut(const Polygon& U, Line l) {
   Polygon ret;
   for(int i = 0; i < U.size(); i++) {
      Point now = U[i], nxt = U[(i + 1) % U.size()];
      if(ccw(l.a, l.b, now) != -1) ret.push_back(now);
      if(ccw(l.a, l.b, now) * ccw(l.a, l.b, nxt) < 0) {
ret.push_back(crosspoint(Line(now, nxt), l)); }
   }
   return (ret);
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_3_A
// 多角形の面積
Real area(const Polygon& p) {
   Real A = 0;
   for(int i = 0; i < p.size(); ++i) { A += cross(p[i], p[(i +
1) % p.size()]); }
   return A * 0.5;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_7_H
// 円と多角形の共通部分の面積
Real area(const Polygon& p, const Circle& c) {
```

```
   if(p.size() < 3) return 0.0;
   function<Real(Circle, Point, Point)> cross_area = [&](const
Circle& c, const Point& a, const Point& b) {
      Point va = c.p - a, vb = c.p - b;
      Real f = cross(va, vb), ret = 0.0;
      if(eq(f, 0.0)) return ret;
      if(max(abs(va), abs(vb)) < c.r + EPS) return f;
      if(distance(Segment(a, b), c.p) > c.r - EPS) return c.r *
c.r * arg(vb * conj(va));
      auto u = crosspoint(c, Segment(a, b));
      vector<Point> tot{a, u.first, u.second, b};
      for(int i = 0; i + 1 < tot.size(); i++) { ret +=
cross_area(c, tot[i], tot[i + 1]); }
      return ret;
   };
   Real A = 0;
   for(int i = 0; i < p.size(); i++) { A += cross_area(c, p[i],
p[(i + 1) % p.size()]); }
   return A;
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_4_B
// 凸多角形の直径(最遠頂点対間距離)
Real convex_diameter(const Polygon& p) {
   int N = (int)p.size();
   int is = 0, js = 0;
   for(int i = 1; i < N; i++) {
      if(p[i].imag() > p[is].imag()) is = i;
      if(p[i].imag() < p[js].imag()) js = i;
   }
   Real maxdis = norm(p[is] - p[js]);

   int maxi, maxj, i, j;
   i = maxi = is;
   j = maxj = js;
   do {
      if(cross(p[(i + 1) % N] - p[i], p[(j + 1) % N] - p[j]) >=
0) {
         j = (j + 1) % N;
      } else {
         i = (i + 1) % N;
      }
      if(norm(p[i] - p[j]) > maxdis) {
         maxdis = norm(p[i] - p[j]);
         maxi = i;
         maxj = j;
      }
   } while(i != is || j != js);
   return sqrt(maxdis);
}

// http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?
id=CGL_5_A
// 最近点対
Real closest_pair(Points ps) {
   if(ps.size() <= 1) throw(0);
   sort(begin(ps), end(ps));

   auto compare_y = [&](const Point& a, const Point& b) {
return imag(a) < imag(b); };
   vector<Point> beet(ps.size());
   const Real INF = 1e18;

   function<Real(int, int)> rec = [&](int left, int right) {
      if(right - left <= 1) return INF;
      int mid = (left + right) >> 1;
      auto x = real(ps[mid]);
      auto ret = min(rec(left, mid), rec(mid, right));
      inplace_merge(begin(ps) + left, begin(ps) + mid,
begin(ps) + right, compare_y);
      int ptr = 0;
      for(int i = left; i < right; i++) {
         if(abs(real(ps[i]) - x) >= ret) continue;
         for(int j = 0; j < ptr; j++) {
            auto luz = ps[i] - beet[ptr - j - 1];
            if(imag(luz) >= ret) break;
            ret = min(ret, abs(luz));
         }
         beet[ptr++] = ps[i];
      }
      return ret;
   };
   return rec(0, (int)ps.size());
}
```

int128.hpp

md5: a4c180

```
std::ostream& operator<<(std::ostream& dest, __int128_t value)
{
   std::ostream::sentry s(dest);
   if(s) {
      __uint128_t tmp = value < 0 ? -value : value;
      char buffer[128];
      char* d = std::end(buffer);
      do {
         --d;
         *d = "0123456789"[tmp % 10];
         tmp /= 10;
      } while(tmp != 0);
      if(value < 0) {
         --d;
         *d = '-';
      }
      int len = std::end(buffer) - d;
      if(dest.rdbuf()->sputn(d, len) != len) {
dest.setstate(std::ios_base::badbit); }
   }
   return dest;
}

__int128 parse(string& s) {
   __int128 ret = 0;
   for(int i = 0; i < s.length(); i++)
      if('0' <= s[i] && s[i] <= '9') ret = 10 * ret + s[i] -
'0';
   return ret;
}
/*
int main() {
  string s = "18782187821878218782187821878218782";
  __int128 x = parse(s);
  x *= 2;
  cout << x << endl;
}
*/
```

random.hpp

md5: bf85b5

```
struct RandomNumberGenerator {
   mt19937 mt;

   RandomNumberGenerator() :
mt(chrono::steady_clock::now().time_since_epoch().count()) {}

   int operator()(int a, int b) {  // [a, b)
      uniform_int_distribution<int> dist(a, b - 1);
      return dist(mt);
   }

   int operator()(int b) {  // [0, b)
      return (*this)(0, b);
   }
};
void solve(int test) {
   RandomNumberGenerator rd;
   cout << rd(0, 1e9) << endl;
   cout << rd(1e5) << endl;
}
```

range_edge_graph.hpp

md5: f3f96c

```
template<typename W> struct range_edge_graph {
   int n;
   struct edge {
      int to;
      W weight;
   };
   vector<vector<edge>> g;

   range_edge_graph(int n) : n(n) {
      g.resize(4 * n);
      for(int i = 1; i < n; ++i) {
         int cl = i << 1 | 0, cr = i << 1 | 1;
         g[i].push_back({cl, 0});
         g[i].push_back({cr, 0});
         g[cl + 2 * n].push_back({i + 2 * n, 0});
         g[cr + 2 * n].push_back({i + 2 * n, 0});
      }
      for(int i = n; i < 2 * n; ++i) g[i].push_back({i + 2 * n,
```

```
0});
    }

    // from [l1, r1) to [l2, r2)
    void add_edge(int l1, int r1, int l2, int r2, W w) {
        int idx = g.size();
        for(l1 += n, r1 += n; l1 < r1; l1 >>= 1, r1 >>= 1) {
            if(l1 & 1) g[l1 + 2 * n].push_back({idx, 0}), l1++;
            if(r1 & 1) --r1, g[r1 + 2 * n].push_back({idx, 0});
        }
        vector<edge> node;
        for(l2 += n, r2 += n; l2 < r2; l2 >>= 1, r2 >>= 1) {
            if(l2 & 1) node.push_back({l2++, w});
            if(r2 & 1) node.push_back({--r2, w});
        }
        g.push_back(node);
    }

    vector<W> dijkstra(int s) {
        s += n;
        vector<W> dist(g.size(), numeric_limits<W>::max());
        dist[s] = 0;
        using P = pair<W, int>;
        priority_queue<P, vector<P>, greater<P>> que;
        que.emplace(0, s);
        while(!que.empty()) {
            P p = que.top();
            que.pop();
            int v = p.second;
            if(dist[v] < p.first) continue;
            for(edge& e : g[v]) {
                if(dist[e.to] > dist[v] + e.weight) {
                    dist[e.to] = dist[v] + e.weight;
                    que.emplace(dist[e.to], e.to);
                }
            }
        }
        vector<W> res(dist.begin() + n, dist.begin() + 2 * n);
        return res;
    }
};
```